

基于物理状态转移方程的过滤器寿命预测与维护策略

摘 要

在工业生产当中，污水需要经过过滤器净化后排放。在此过程中，污水过滤器的透水率会发生变化，下降到一定数值就无法使用，因此工厂需要对设备进行维护。依据某工厂两年过滤器透水率的实时监测情况，本文主要研究回应透水率变化与过滤器寿命的规律、使得工厂污水净化年成本最小的过滤器维护方案。通过 STL 时间序列分解、物理状态转移方程、遗传算法等进行求解。

针对问题一：采用改进的自适应 Hampel 滤波器进行数据清洗，通过两阶段检测策略有效剔除了异常值并保留了维护引发的真实阶跃信号，使用线性插值填补缺失值减小数据偏差。通过 STL 时间序列分解得出透水率以一年为周期的波动特征，采用快速傅里叶变换与自相关系数验证结论。利用季节影响算子、劣化斜率得出透水率夏季劣化慢、秋冬季劣化快，且劣化速度随使用年限加快的下降规律。引入维护增益与维护损伤系数，量化不同维护类型的瞬时恢复效果和长期损伤累积。最后提取了刻画透水率变化的四项核心指标：**基准劣化率**反映无维护干预下过滤器的自然衰减速度；**季节影响算子**量化季节因素对透水率的影响；**维护增益**描述不同维护操作对透水率的瞬时提升效果；**维护损伤系数**则衡量维护活动对过滤器长期劣化速率的加速效应。

针对问题二：基于提取的四个核心指标，构建透水率物理状态转移方程 $x(t+1) = x(t) - D(t) + G(t)$ ，利用预处理的数据拟合不同设备的参数，建立基于 365 天滑动平均透水率的寿命终止判定标准。随后建立回溯方程逆向回溯还原设备出厂初始透水率，并正向预测各过滤器寿命，10 台设备预测寿命在 4.65-7.45 年。最后通过均方根误差 (RMSE)、平均绝对百分比误差 (MAPE) 进行模型可靠性检验。

针对问题三：首先建立使污水净化的年均成本最小的优化模型，由优化函数、决策变量、约束条件组成。首先用购买成本加上维护成本表示过滤器一个生命周期的成本，再从一个生命周期折算至一年的成本得到**优化函数**： $\min J(X) = \frac{C_{\text{purchase}} + C_{\text{mid}} \cdot n_{\text{mid}} + C_{\text{big}} \cdot n_{\text{big}}}{L}$ 。接着针对过滤器维护需求，制定 3 个**决策变量**：基于最大维护周期的需求，定义中维护间隔阈值 T_{mid} 与大维护间隔阈值 T_{big} ；基于透水率退化速率下降速率过快需要维护的需求，引入退化速率触发阈值 r_{th} 。随后根据工程实际，对过滤器的运行状态、退化规律及维护触发逻辑施加**约束条件**。最后，基于动态系统约束是非线性的、目标函数不可解析表达等特点，使用**遗传算法**制定维护策略，并使用训练集与测试集验证算法可靠性。

针对问题四：考虑到在市场过滤器购买价格与维护成本会出现波动，需要改变相应参数进行优化策略的灵敏度分析。通过对两类成本分别施加 $\pm 10\%$ 和 $\pm 15\%$ 的扰动，并定义**损失率** δ 为固定原策略下的年均成本相对于新成本下重新优化所得年均成本的偏差比例，以衡量第三问所建策略在新成本环境下的表现。结果表明，所有情景下损失率绝对值均**不超过 3.34%**，验证了建立的维护策略的合理性以及模型的普适性与稳定性。

关键字：污水过滤器 Hampel 滤波器 STL 时间序列分解 遗传算法 状态转移方程

一、 问题重述

1.1 问题背景

工厂生产过程中的污水若要排放需要经过过滤以达到净化标准，对过滤器的使用和维护也即成为了一个关键的问题。透水率是研究过滤器性能的重要指标，通过透水率可以看出过滤器的状态与性能。随着过滤器的使用，过滤物附着在过滤器上会使过滤器的透水率降低，此时就需要维护，当维护无法改善过滤器的透水状态时，则需要更换过滤器。维护分为 3-5 天的日常小维护，透水率下降较快时或维护间隔较长时的中维护，和每年 0-4 次的大维护。但维护也会对过滤器造成性能上的损伤，且过滤效果与维护效果受季节影响。

1.2 问题提出

某工厂从 2022 年 4 月开始使用了 10 台过滤器。附件 1 给出了这 10 台过滤器在 2024 年 4 月-2026 年 4 月的透水率检测数据，附件 2 给出了这 10 台过滤器 2 年内中维护和大维护的记录。现需结合实际情况与附件所给信息建立数学模型，分析以下问题：

问题一：为了研究数据特征，了解影响过滤器透水率的因素，需要：

1. 对附件的缺失值与异常值进行对应处理
2. 分析数据的周期性，下降趋势规律，以便探索过滤器透水率的特征，利于后续的采购、维护与更换等安排
3. 探究不同维护类型对透水率的影响
4. 给出造成透水率变化的重要指标

问题二：给出过滤器寿命的预测模型，需要综合考虑以下因素：

1. 季节变化
2. 过滤器性能的下降规律
3. 各个维护类型对透水率的影响

最后使用构建的模型预测 10 台过滤器的寿命。

问题三：以污水净化年均成本最小为优化目标，给出最优的维护方案。

问题四：通过改变过滤器价格与维护成本对模型进行灵敏度分析，验证模型在不同市场情况下的稳健性。

二、 问题分析

2.1 问题一分析

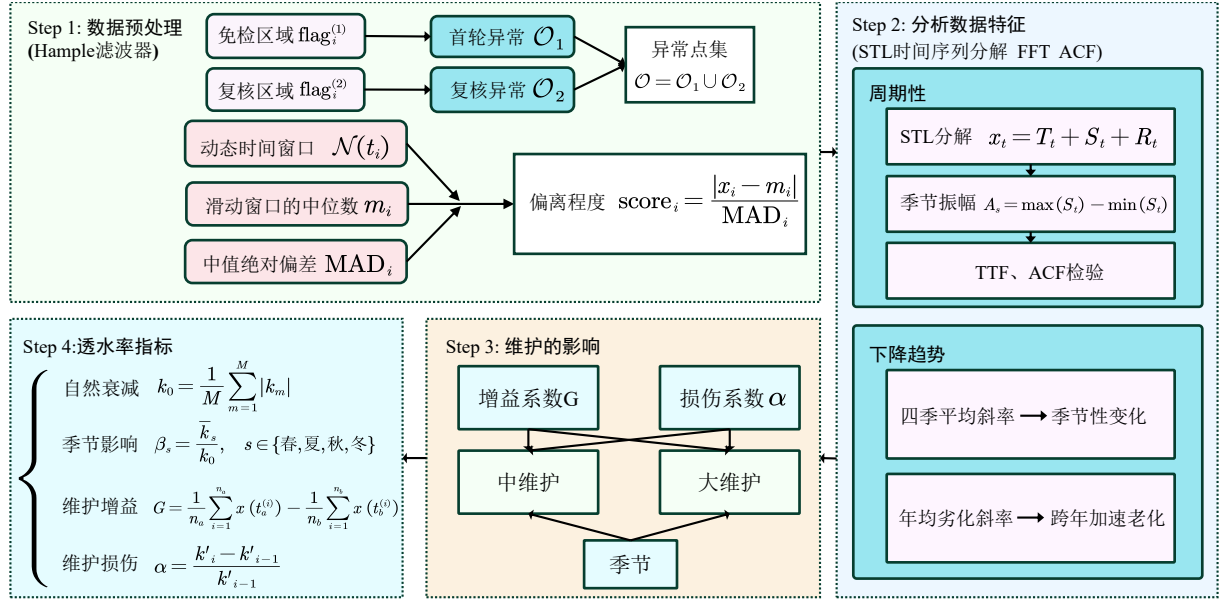


图 1 问题一分析流程图

首先对问题进行拆分，问题一主要有 4 方面的要求：对数据预处理、分析数据的特征以及规律、探究维护类型对透水率的影响、给出影响透水率的重要指标。以下为解决 4 个需求的方法分析：

1. 数据预处理：

对于缺失值，按成因分为两类处理：在中维护和大维护期间，因维护操作导致的数据缺失，不进行填补，以保留原始无记录状态；非维护期间的缺失，若位于数据起始阶段则直接剔除，其他如适配传感器临时故障等因素造成的短期缺失，采用多段线性插值填补，既能保持时间序列的整体趋势，又不会引入明显偏差。

对于异常值，我们需要在保留维护引起的阶跃变化的前提下，剔除其他随机异常波动。首先依据维护事件标记保护期，将保护期内数据直接视为正常值且不参与检测；随后对非保护期数据使用 Hampel 滤波器逐点统计检测并标记异常点。对非维护期出现的短期连续异常，结合现场记录判断是否为持续性堵塞或故障，确属异常的追加标记。全部标记完成后，对异常点使用线性插值做替换处理。

2. 分析数据特征：

由于想知道数据在时间维度上的性质，我们通过 STL 时间序列拆分得到了过滤器透水率的季节振幅，从而得到透水率的周期规律，随后通过 FFT 与 ACF

指标验证了规律的正确性。

将下降趋势规律拆分为一年内的季节性变化和年与年之间的周期变化。通过 STL 时间序列拆分得到的季节振幅，结合设置的季节影响算子，得出透水率在不同季节的下降规律。又通过对比不同年份同一季节的下降趋势可以得到过滤器随着时间推移下降趋势的规律。

3. 维护类型对透水率的影响：

在附件 1 的数据中看出小维护对透水率的改善有限，又因小维护成本不计，于是我们主要研究中维护与大维护对透水率的影响。通过引入维护增益与维护损伤系数判断每次中维护与大维护对过滤器性能的影响。

4. 造成透水率变化的指标：

针对过滤器对透水率的的自然衰减、季节对透水率的影响、维护对过滤器造成的不可逆的损伤、维护对透水率的增加 4 个关键影响，分别建立基准劣化率、季节影响算子、维护损伤系数、维护增益 4 个指标。指标可以通过附件 1 的数据拟合、列式计算得到。

2.2 问题二分析

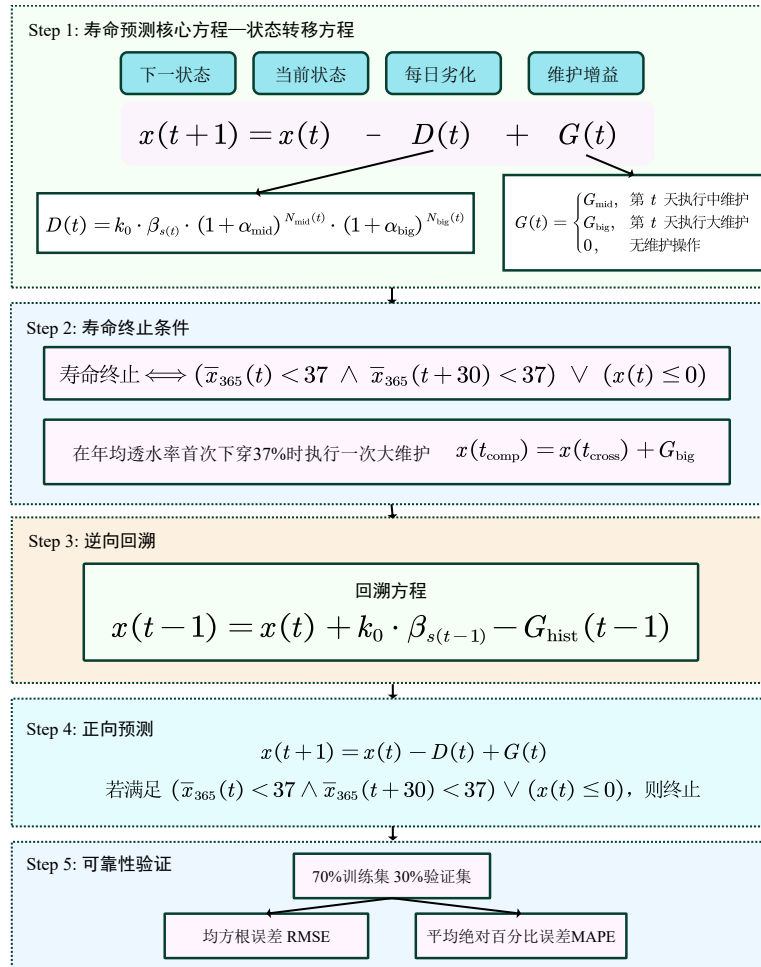


图 2 问题二分析流程图

为了综合考虑过滤器的性能的自然下降、季节对过滤器性能的影响、维护对透水率的影响，将第一问中的 4 个指标相互关联，并纳入第二问的寿命预测模型，即状态转移方程。将模型分别套用至每个过滤器，得到每台过滤器的各项指标以及固定维护规律。当过去以年的平均透水率低于 37 时，且维护后一个月以内又下降至 37 及以下时，判定为过滤器寿命终止。随后建立逆向回溯方程，回溯过滤器在出厂状态时的透水率，再正向预测至寿命终止。最后给出 10 台过滤器的寿命预测结果，设置训练集与测试集并使用 RMSE 与 MAPE 进行模型可靠性验证。

2.3 问题三分析

基于前两问对过滤器劣化的规律以及维护对透水率的影响，建立优化模型：

1. **优化函数：**通过预测模型预测出的一个过滤器从开始使用到寿命终止的一个完整使用周期，随后计算出整个使用周期的成本，再通过一个使用周期的成本换算到一年的成本。以年均成本最小为优化函数，
2. **决策变量：**考虑到透水率下降速率与距上次维护时间均影响维护需求，设立中维护时间间隔阈值 T_{mid} 与退化速率触发阈值 r_{th} 两个决策变量，实现固定间隔与状态响应的混合调度；考虑到过滤器长期运行后性能严重衰减，设立大维护时间间隔阈值 T_{big} 决策变量，控制大维护执行频率。
3. **约束条件：**考虑到过滤器实际运行的物理约束，从四个方面建立约束条件：透水率始终为正、365 天滑动平均透水率不低于 37、透水率演化服从第二问状态转移方程、维护触发逻辑由决策变量按规则控制。

最后根据本题动态系统约束是非线性的、维护行为会导致状态跳变、目标函数不可解析表达的特点，选择使用遗传算法制定维护策略。

2.4 问题四分析

改变过滤器购买价格与维护成本验证维护策略的稳健性，也就是对过滤器购买价格与维护成本进行敏感性分析。于是需要设置题目给的过滤器价格与维护价格波动。为了量化建立的策略在新参数下的表现，可以设损失率 δ 为固定策略不变，年均成本相对于新成本下重新优化所得年均成本的偏差比例，并用它进行敏感性分析。

三、 模型假设

1. 假设小维护对透水率改善有限且成本可忽略不计。附件数据显示小维护频率高但单次效果微弱，且题目明确其成本可忽略，因此我们只研究中维护与大维护的影响。

2. 假设维护操作在维护日当天立即生效。实际维护后透水率会在较短时间内恢复，为简化模型，假设增益在维护当日一次性完成。
3. 假设季节对透水率下降速率的影响按月离散变化，不同月份对应不同的影响系数。季节变化对过滤器性能的影响在月份交界处变化显著，且数据难以支持连续函数拟合，故采用离散跳变的季节算子进行建模。
4. 假设未来气候与运行条件保持稳定。缺乏未来年份的环境预测数据，故假设历史规律可外推。

四、 符号说明

符号	说明	单位
$x(t)$	第 t 天的透水率	—
$\bar{x}_{365}(t)$	前 365 天滑动平均透水率	—
k_0	基准劣化率	—/天
β_s	季节影响算子	—
G_{mid}	中维护瞬时增益	—
G_{big}	大维护瞬时增益	—
α_{mid}	中维护损伤系数	—
α_{big}	大维护损伤系数	—
$N_{\text{mid}}(t)$	截至第 t 天的中维护累计次数	—
$N_{\text{big}}(t)$	截至第 t 天的大维护累计次数	—
A_s	季节振幅	—
T_t	STL 分解趋势分量	—
S_t	STL 分解季节分量	—
R_t	STL 分解残差分量	—
C_{purchase}	过滤器购买价格	万元
c_{mid}	单次中维护成本	万元
c_{big}	单次大维护成本	万元
L	过滤器总寿命	年
J	年均成本	万元/年
T_{mid}	中维护时间间隔阈值	天
T_{big}	大维护时间间隔阈值	天
r_{th}	退化速率触发阈值	—/天
δ	损失率	%

五、 问题一的模型的建立和求解

我们围绕数据预处理、数据特征与规律分析、维护类型对透水率的影响探究，以及关键影响指标识别四个方面，分别开展建模与求解工作。

5.1 利用 Hampel 滤波器进行数据预处理

针对工业污水过滤器透水率数据存在随机异常波动、缺失值、维护引发阶跃变化且采样间隔不均匀的特点，采用带维护事件标记的改进自适应 Hampel 滤波器完成数据预处理。该方法基于动态时间窗口与绝对中位差构建稳健识别规则，采用两阶段检测策略，既能剔除随机噪声与异常点，又能完整保留维护操作带来的真实阶跃信号，适配透水率长期衰减、受季节与维护共同影响的监测场景^[1]。

5.1.1 异常点的判断方法

我们采用动态时间窗口 $\mathcal{N}(t_i)$ 对数据进行分段处理，以时间替代固定点数来划分判断该点是正常点或异常点的半径：

$$\mathcal{N}(t_i) = \{x_j \mid |t_j - t_i| \leq \Delta T_{\max}, j \neq i\}, \quad (1)$$

其中： x_j 为时间点 t_j 处的透水率测量值， $\Delta T_{\max} = 72$ 小时为最大时间搜索半径。该设计使滤波器能够适应数据采样间隔不均匀的实际工况。

为保证窗口内有足够的点来判定该点是否为异常值，设置窗口内需要至少有 5 个点，即添加最低约束：

$$|\mathcal{N}(t_i)| \geq N_{\min}, \quad N_{\min} = 5. \quad (2)$$

采用滑动窗口的中位数 m_i 作为该点正常值的基准：

$$m_i = \text{median} \{x_j \mid j \in \mathcal{N}(t_i)\}. \quad (3)$$

采用中值绝对偏差 MAD_i 来作为基于正常值的波动范围：

$$\text{MAD}_i = \text{median} \{|x_j - m_i| \mid j \in \mathcal{N}(t_i)\}. \quad (4)$$

使用异常评分函数 score_i 判断偏离正常值的尺度：

$$\text{score}_i = \frac{|x_i - m_i|}{\text{MAD}_i}. \quad (5)$$

与传统 Hampel 滤波器不同，我们不使用 1.4826 缩放系数，直接以原始 MAD 作为尺度估计，使异常判定标准更为严格，避免透水率数据中异常波动的漏检。

5.1.2 两阶段自适应检测策略

维护操作引发的透水率阶跃变化属于真实物理信号，不应被滤波器误判为异常。为此，本文构建两阶段自适应检测策略，在保护维护信号的同时识别真正的异常点。

基于维护事件标记，定义第一阶段免检区域为维护日与其之后的 24 小时：

$$\text{flag}_i^{(1)} = \begin{cases} 1, & t_i \in \text{维护日} \cup (\text{维护日} + 24\text{h}) \\ 0, & \text{其他}. \end{cases} \quad (6)$$

第一轮检测仅对免检区域外的数据点进行异常判定：

$$\mathcal{O}_1 = \left\{ i \mid \text{score}_i > T, \text{flag}_i^{(1)} = 0, |\mathcal{N}(t_i)| \geq N_{\min} \right\}, \quad T = 3. \quad (7)$$

为避免第一轮异常点对后续检测造成干扰，采用线性插值对其进行临时填充，得到中间序列：

$$\tilde{x}_i = \begin{cases} x_i, & i \notin \mathcal{O}_1, \\ \text{interp}(t_i), & i \in \mathcal{O}_1. \end{cases} \quad (8)$$

第一轮检测中被免检的点，即在维护后 24 小时内的点，也可能有异常点，需进行第二轮复核。第二轮检测区域为：

$$\text{flag}_i^{(2)} = \begin{cases} 1, & t_i \in (\text{维护日} + 24\text{h}) \setminus \text{维护日}, \\ 0, & \text{其他}. \end{cases} \quad (9)$$

为确保复核的独立性，第二轮检测的参考窗口排除维护日当天的数据点：

$$\mathcal{N}^{(2)}(t_i) = \left\{ j \mid |t_j - t_i| \leq \Delta T_{\max}, j \neq i, \text{flag}_j^{(\text{day})} = 0 \right\}. \quad (10)$$

第二轮异常检测规则为：

$$\mathcal{O}_2 = \left\{ i \mid \text{score}_i^{(2)} > T, \text{flag}_i^{(2)} = 1, |\mathcal{N}^{(2)}(t_i)| \geq N_{\min} \right\}. \quad (11)$$

最终异常点集合取两轮检测结果的并集：

$$\mathcal{O} = \mathcal{O}_1 \cup \mathcal{O}_2. \quad (12)$$

5.1.3 滤波后的缺失值处理

最后对异常值进行处理，使用线性插值填补，能保留时间序列趋势，不会引入大偏差：

$$\hat{x}_i = x_{i_L} + \frac{x_{i_R} - x_{i_L}}{t_{i_R} - t_{i_L}} \cdot (t_i - t_{i_L}). \quad (13)$$

其中 $\hat{x}_i$ 为待插值点 t_i 处的估计值， t_{i_L} 和 t_{i_R} 分别为 t_i 前后最近有效数据点的时间

坐标, x_{i_L} 和 x_{i_R} 为对应时间点的透水率测量值, $\frac{x_{i_R}-x_{i_L}}{t_{i_R}-t_{i_L}}$ 为两点间的斜率。

维护日及维护后 24 小时的原始缺失值保持为 NaN, 不进行插值, 最终处理得到的值为:

$$x_i^{\text{clean}} = \begin{cases} \text{NaN}, & i \in \text{维护期} \cap \text{原始缺失}, \\ \hat{x}_i, & i \in \mathcal{O} \cup \text{原始缺失}, \\ x_i, & \text{其他}. \end{cases} \quad (14)$$

对非维护期内存在的缺失值, 同样采用 (13) 式的线性插值方法进行填补。

5.1.4 数据预处理结果

采用改进的自适应 Hampel 滤波器对原始透水率数据进行清洗, 有效剔除异常波动并填补缺失值。以设备 A5 和 A6 为例, 清洗前后的透水率序列对比如图 3 和图 4 所示:

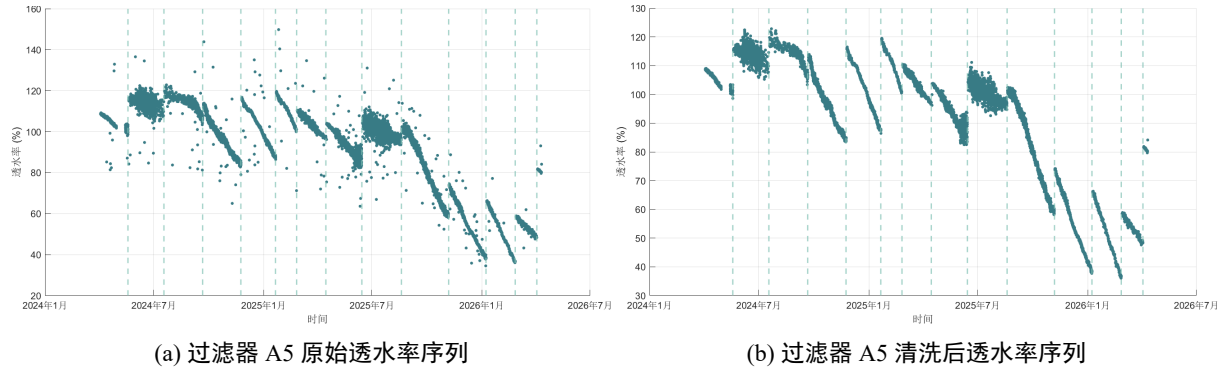


图 3 过滤器 A5 数据清洗前后对比

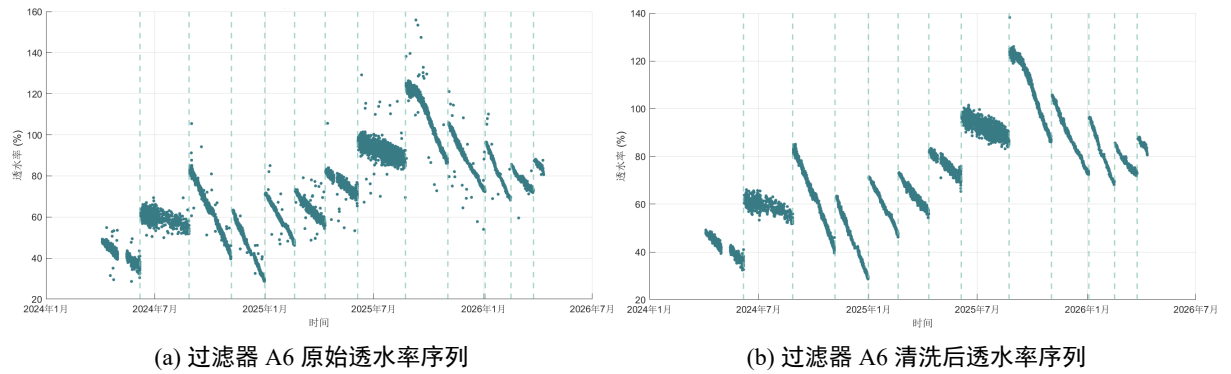


图 4 过滤器 A6 数据清洗前后对比

从图中看出, 经过改进的自适应 Hampel 滤波处理后, 透水率序列中的异常跳变点被有效剔除, 缺失值通过线性插值得到合理填补, 序列整体趋势更加平滑、连续且稳定, 为后续的劣化特性分析与寿命预测模型构建提供了高质量的数据基础。

5.2 分析数据特征

接下来分别分析数据的周期性、下降趋势规律。

5.2.1 数据的周期性

透水率作为反映过滤器性能与状态的核心指标，其变化受环境温度、水质条件等季节性因素影响显著。为识别数据中隐含的周期波动，采用 STL（Seasonal-Trend decomposition using Loess）分解法对每台过滤器的日均透水率序列进行拆解^[2]。

STL 方法基于局部加权回归，将原始序列分解为三个可解释的分量：

$$x_t = T_t + S_t + R_t, \quad (15)$$

其中 x_t 是第 t 日的透水率， T_t 是趋势分量，反映过滤器长期老化导致的透水率单调下降， S_t 是季节分量，反映以一年为周期的规律性波动， R_t 是残差分量，代表随机噪声或未捕获的细微扰动。三个分量的具体构造方式如下：

- **趋势项 T_t ：**采用滑动平均对原始数据做平滑处理，滤除短期波动和周期波动，体现曲线的大致走向，表示过滤器因老化导致的透水率逐年下滑趋势。
- **季节项 S_t ：**从原始数据中减去趋势项得到，随后对每个周期位置分别取平均，形成一个固定的年度波形。为使季节分量在各周期之间可比，进一步对该波形进行中心化处理，使其在一个完整周期内的均值为零，即 $\sum_{t=1}^{365} S_t = 0$ 。最终结果呈现为每年重复的波动形态。
- **残差项 R_t ：**定义为原始序列减去趋势项与季节项后的剩余部分，即 $R_t = x_t - T_t - S_t$ ，代表随机噪声或细微扰动。

上述分解过程通过 MATLAB 内置函数 `trenddecomp` 实现，调用格式为：

```
1 [trend, seasonal, residual] = trenddecomp(daily_per_full, 'stl', 365);
```

其中 `daily_per_full` 为日均透水率序列，`'stl'` 指定 STL 分解方法，365 把一年设置为周期参数。

我们选择过滤器 A2 来展示 STL 的分析结果，即 (15) 式的 4 个变量：

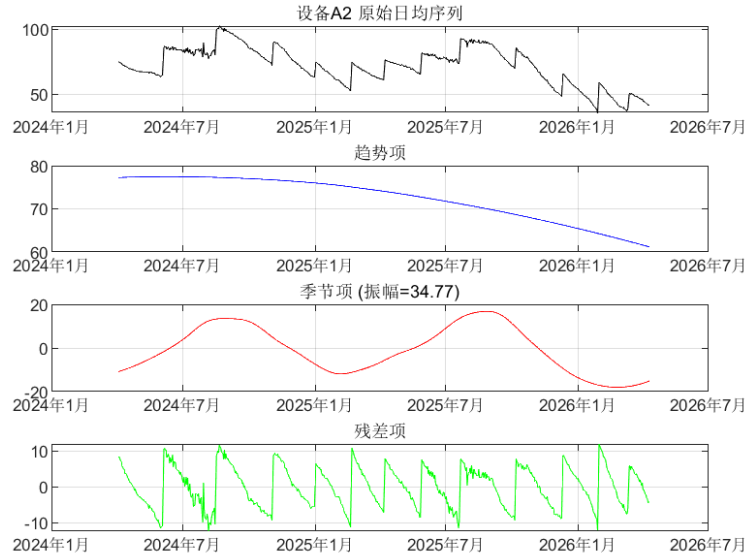


图 5 A2 过滤器的 STL 参数

从图 5中可以看出：

- 原始序列呈现出明显的“冬季低、夏季高”的年度波动形态
- 趋势分量剥离季节影响后，透水率整体呈缓慢下降趋势，符合过滤器老化规律
- 季节分量展现出稳定的周期性波形，每年重复相似的变化模式
- 残差分量幅值较小且随机分布，表明分解充分提取了主要信号

为量化季节效应的强度，我们用季节分量 S_t 定义季节振幅 A_s ：

$$A_s = \max(S_t) - \min(S_t) \quad (16)$$

它代表一年内纯季节因素引起的透水率最大波动幅度。

接着挑选了 A1-A5 过滤器重点分析季节性分量以及季节振幅以得到周期性规律。

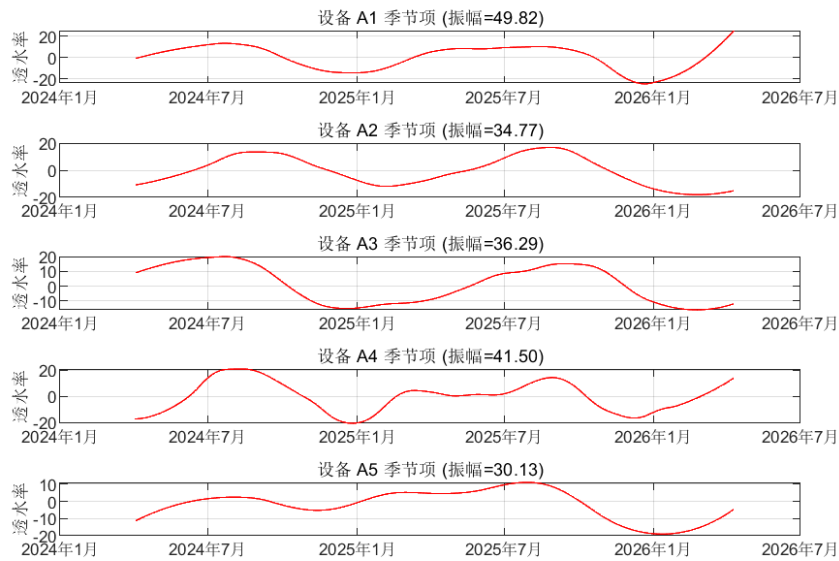


图 6 过滤器 A1-A5 的季节分量变化

由图 6 看出季节分量以一年为周期进行波动，说明过滤器的透水率存在以一年为周期的波动特征。10 台过滤器的季节振幅结果如表 1 所示。

表 1 各过滤器季节振幅统计

过滤器	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
振幅 A_s	49.82	41.07	45.18	37.81	32.94	42.75	27.09	31.46	35.01	51.87

10 台过滤器的季节振幅介于 27.09 至 51.87 之间，平均值为 38.52。注意到不同过滤器对季节变化的敏感程度存在差异，比如 A10、A1 等过滤器振幅较大，说明其性能受季节影响更为显著，A7、A5 等过滤器振幅相对较小，季节干扰相对平缓。

随后采用快速傅里叶变换（FFT）对去趋势后的序列进行频域分析，并绘制自相关函数（ACF）图，以验证结论。

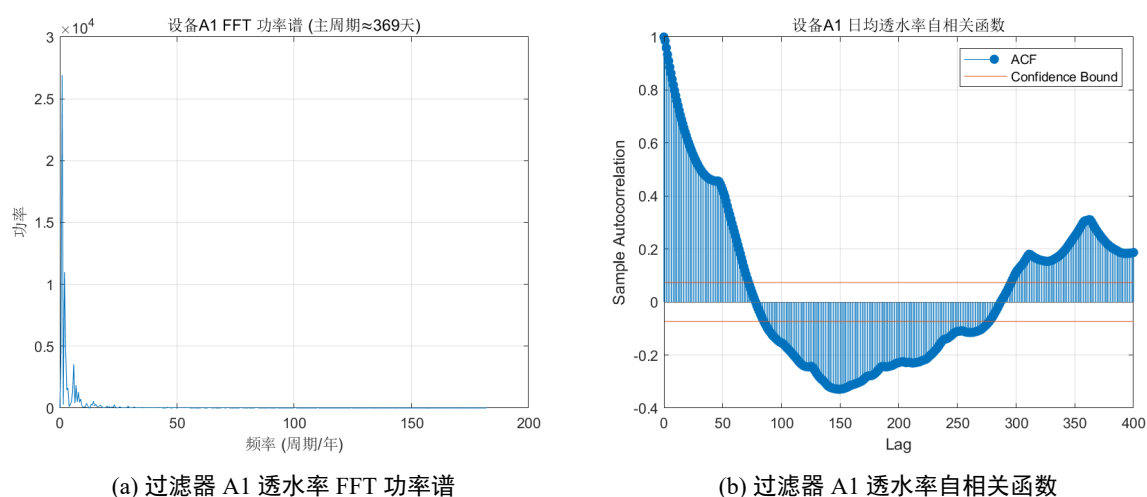


图 7 过滤器 A1 透水率序列的周期性分析（FFT 与 ACF）

由图 7，过滤器 A1 的 FFT 功率谱与自相关函数（ACF）共同验证了其日均透水率的周期性：

- 由图 7a，FFT 功率谱图以频率为横轴、功率强度为纵轴，识别序列中的主导周期成分。图中在频率 $x = \frac{1}{369}$ 处出现显著主峰，对应主周期约 369 天，接近一年的周期，且无其他高频强周期，表明透水率存在强烈的年周期性波动。
- 由图 7b，自相关函数图以滞后阶数（天）为横轴、样本自相关系数为纵轴，用于衡量序列与其滞后项的相关性。图中在滞后约 350-380 天处呈现明显的正相关峰值，且超出置信界，印证了透水率序列存在约一年的周期波动；同时，在 0-150 天左右，自相关系数从 1 快速衰减，反映了序列的短期强自相关性，说明透水率变化具有连续性，受之前透水率影响显著。

综合 STL 分解、频谱分析与自相关检验，得到过滤器的透水率变化是以年为周期的。

5.2.2 下降趋势规律

我们以时间为分隔，研究一年之间的下降趋势规律，还有年与年之间的下降变化规律。前者的变化主要来源于季节的影响，后者的变化主要来源于维护与过滤器自然老化的影响。

一年内季节下降规律 将各维护间隔内的日均劣化斜率按季节归类，分别计算春、夏、秋、冬四季的平均斜率。第 m 个维护间隔内的劣化斜率 k_m 通过对该时段内的日均透水率序列进行线性拟合得到：

$$k_m = \frac{n \sum_{i=1}^n t_i x_i - \sum_{i=1}^n t_i \sum_{i=1}^n x_i}{n \sum_{i=1}^n t_i^2 - (\sum_{i=1}^n t_i)^2}, \quad (17)$$

其中 t_i 为间隔内的第 i 天， x_i 为对应日的透水率， n 为该间隔内的有效天数。

将各间隔按其时间中点的月份归入对应季节，分别计算各季节内斜率的平均值：

$$\bar{k}_s = \frac{1}{M_s} \sum_{m \in \mathcal{S}_s} k_m, \quad s \in \{\text{春, 夏, 秋, 冬}\}, \quad (18)$$

其中 $\mathcal{S}_s$ 为属于季节 s 的间隔索引集合， M_s 为集合大小。

经过计算，春、夏、秋、冬四季的平均斜率结果如表 2 所示。

表 2 各过滤器四季劣化斜率

过滤器	春季斜率	夏季斜率	秋季斜率	冬季斜率
A1	-0.2694	-0.1718	-0.4886	-0.4828
A2	-0.2512	-0.1628	-0.4997	-0.5248
A3	-0.2063	-0.1620	-0.5862	-0.6186
A4	-0.1854	-0.1589	-0.5063	-0.4766
A5	-0.2445	-0.1342	-0.5245	-0.5588
A6	-0.2754	-0.1024	-0.5919	-0.5745
A7	-0.2471	-0.1518	-0.5606	-0.5064
A8	-0.2891	-0.1275	-0.6277	-0.5128
A9	-0.2268	-0.1183	-0.4454	-0.5487
A10	-0.2737	-0.1576	-0.6338	-0.6250
平均	-0.2469	-0.1447	-0.5465	-0.5429

从四季平均斜率来看，夏季劣化最慢，平均斜率为-0.1447 单位/天；秋季和冬季劣化最快，平均斜率分别为-0.5465 和-0.5429 单位/天，约为夏季的 3.8 倍；春季介于两者之间，平均斜率为-0.2469 单位/天。该规律符合流体的物理特性：夏季水温高、水体粘度低，流体通过滤芯的阻力较小，故透水率下降缓慢；秋冬季水温降低、水体粘度增大，同时水中悬浮物浓度升高、微生物活性变化，导致滤芯堵塞加速，透水率快速衰减。

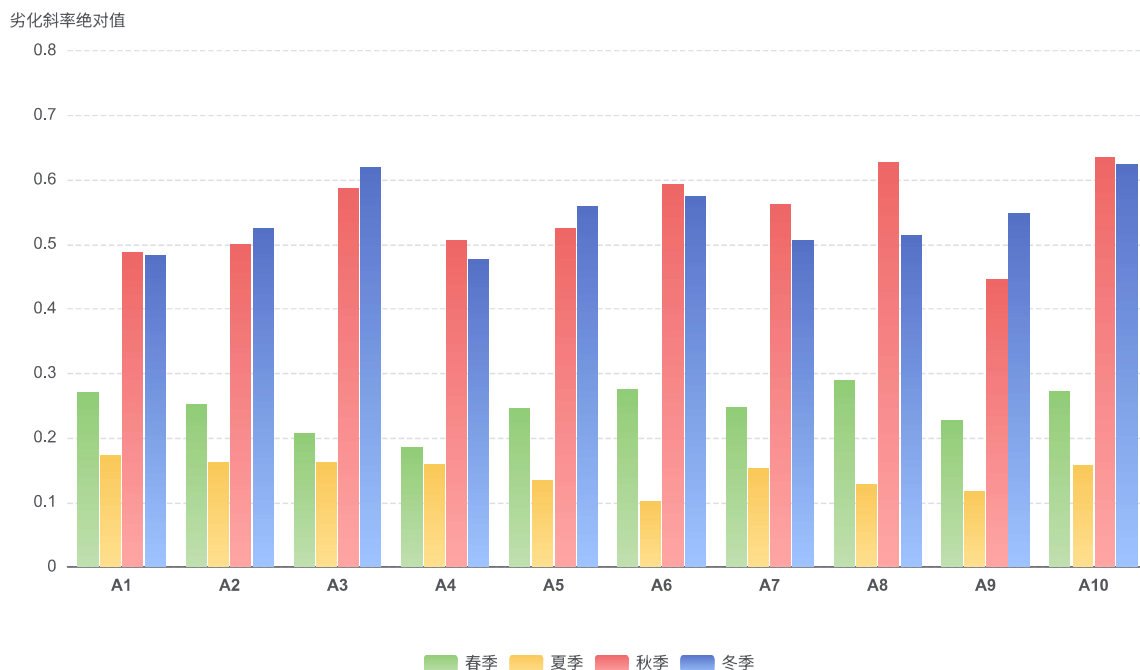


图 8 各过滤器四季劣化斜率对比

由图 8，10 台过滤器的透水率劣化速率呈现出高度一致的季节性特征：秋冬季劣化速率最快，斜率绝对值显著高于春夏季；夏季劣化速率最慢，劣化进程明显放缓；春季则表现为过渡性特征。

跨年份下降规律 为考察过滤器性能随使用年限的退化趋势，按年度统计各过滤器的年均劣化斜率，结果如表 3 所示。

表 3 各过滤器年度劣化斜率

年度	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
2024.4-2025.3	-0.2919	-0.3138	-0.3586	-0.3268	-0.3131	-0.3800	-0.3155	-0.3489	-0.3476	-0.3901
2025.4-2026.4	-0.3790	-0.3870	-0.3710	-0.3365	-0.3882	-0.3977	-0.3834	-0.3801	-0.3385	-0.4111

对比两个年度数据，除 A3 和 A4 外，其余 8 台过滤器的劣化斜率绝对值均呈现不同程度的增大，即透水率下降速度随时间加快。其中 A2 从-0.3138 增至-0.3870，增幅达 23.3%；A5 从-0.3131 增至-0.3882，增幅达 24.0%；A10 从-0.3901 增至-0.4111，增幅为 5.4%。这表明随着使用年限增加，过滤器的自然老化与维护累积损伤共同作用，导致过滤器加速老化。

综合上述分析，透水率下降呈现出季节周期性与跨年度加速老化的特征。季节周期性为维护时间提供了建议：因为冬天透水率下降快，可以考虑在冬季维护。跨年度加速老化的特征帮助工厂制定维护或更换方案。

5.3 维护类型对透水率的影响

维护操作对透水率产生两方面影响：一是维护后透水率的瞬时提升幅度，即维护增益；二是维护对后续劣化速度的长期影响，即维护损伤。我们分别分析增益与损伤对中维护与大维护的作用。

5.3.1 维护增益分析

维护增益定义为维护前后各 3 天透水率均值之差：

$$G = \frac{1}{n_{\text{after}}} \sum_{i=1}^{n_{\text{after}}} x(t_{\text{after}}^{(i)}) - \frac{1}{n_{\text{before}}} \sum_{i=1}^{n_{\text{before}}} x(t_{\text{before}}^{(i)}),$$

其反映单次维护的即时恢复能力。各过滤器中维护与大维护的平均增益统计结果如表 4 所示。

表 4 各过滤器维护增益统计

过滤器	中维护增益	大维护增益	增益差值
A1	16.40	7.72	8.69
A2	12.48	12.82	-0.35
A3	12.34	10.28	2.07
A4	16.25	—	—
A5	13.19	3.96	9.23
A6	18.22	16.66	1.57
A7	10.43	16.59	-6.16
A8	10.70	—	—
A9	13.95	12.89	1.05
A10	11.03	8.12	2.91
平均	13.50	11.13	2.37

从整体水平来看，10 台过滤器中维护平均增益为 13.50，大维护平均增益为 11.13，中维护短期恢复效果平均高出约 2.37 个透水率单位。中维护增益普遍较高且稳定，介于 10.43 至 18.22 之间；大维护增益波动较为剧烈，介于 3.96 至 16.66 之间。A1、A5 等过滤器的大维护增益远低于中维护，反映出大维护时过滤器可能已处于深度劣化状态，即使彻底清洗也难以恢复到较高水平。值得注意的是，A2 和 A7 的大维护增益反超中维护，这可能是因为大维护时机恰好处于透水率极低点，且大维护清洗更为彻底，使得恢复幅度反而更大。

结合现实分析，中维护以清除近期附着物为主，能快速释放被堵孔道，因而瞬时增益较高。大维护虽能更深度清理，但其透水率已大幅衰减，可恢复的绝对空间减小，导致维护增益反而不大。

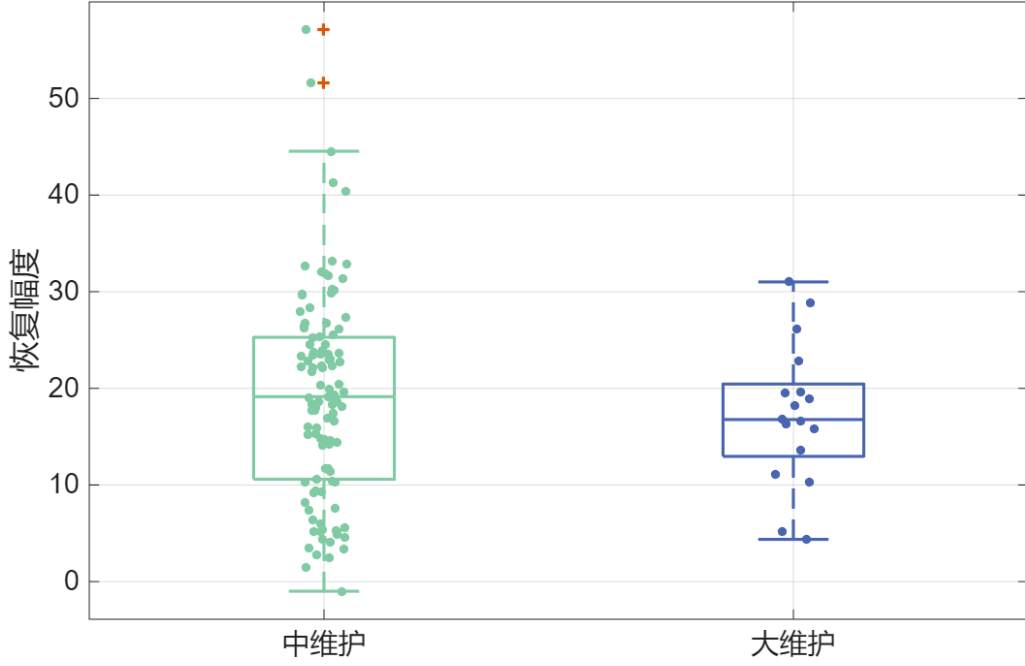


图 9 不同维护类型恢复幅度对比

由图 9，中维护的透水率恢复幅度整体略高于大维护，其中位数约为 20%，且分布更分散，存在多个超过 40% 的高恢复值；大维护的恢复幅度则更集中稳定，中位数约为 17%，整体波动范围较小，二者共同反映出中维护在部分工况下的瞬时恢复能力更强，而大维护的恢复效果更具一致性。

5.3.2 维护损伤分析

纳入维护损伤系数 α 量化维护操作对过滤器劣化速度的影响，用剔除季节因素后相邻段纯净劣化率的相对变化率来表示，先剔除季节因素：

$$k'_i = \frac{|k_i|}{\beta_{s(i)}},$$

其中 k_i 为第 i 段的有量纲斜率, $\beta_{s(i)}$ 为该段所属季节的季节算子 $\beta_s = \frac{\bar{k}_s}{k_0}$, $s \in \{\text{春, 夏, 秋, 冬}\}$, k_0 取斜率的绝对值后平均 $k_0 = \frac{1}{M} \sum_{m=1}^M |k_m|$ 。则损伤系数定义为相邻段纯净劣化率的相对变化率：

$$\alpha = \frac{k'_i - k'_{i-1}}{k'_{i-1}},$$

正值表示劣化加速，负值表示劣化减缓。我们把负值都填充为 0，因为维护之后的一段时间理论上透水率下降应该更快，在模型中负值属于不合理的值。各过滤器损伤系数统计如表 5 所示。

表 5 各过滤器维护损伤系数统计

过滤器	中维护损伤系数	大维护损伤系数	损伤系数差值
A1	0.0554	0.1166	-0.0613
A2	0.1967	0.0000	0.1967
A3	0.1262	0.0890	0.0372
A4	0.2270	0.0000	0.2270
A5	0.0178	0.0000	0.0178
A6	0.0181	0.1773	-0.1593
A7	0.2082	0.0000	0.2082
A8	0.2494	0.0000	0.2494
A9	0.0281	0.3995	-0.3714
A10	0.0570	0.7481	-0.6911
平均	0.1184	0.1531	-0.0347

中维护损伤系数平均值为 0.1184，大维护损伤系数平均值为 0.1531，二者差异不大。值得注意的是，过滤器 A4、A8 由于没有大维护记录，大维护损伤系数为 0；过滤器 A2、A5、A7 的大维护损伤原始值为负，截断后变为 0，意味着大维护往往能使维护后的斜率变得更平缓，即大维护能有效减缓短期劣化速度，但一段时间后老化速率可能会增加。而中维护由于清洗不彻底，残留垢层可能导致局部污堵加速，使得损伤系数多为正值。大维护的真实损伤主要体现在永久峰值衰减上，而非劣化速度的加速。

5.3.3 季节对维护增益的影响

季节因素对维护增益具有显著影响。按维护发生所在季节统计中维护的平均增益，结果如表 6 所示。

表 6 中维护增益季节分布统计

过滤器	春季	夏季	秋季	冬季
A1	17.23	7.51	9.81	30.25
A2	10.82	14.65	12.31	11.78
A3	10.96	4.58	13.34	15.78
A4	18.24	14.55	9.17	21.33
A5	12.84	7.28	16.66	16.15
A6	13.72	21.08	11.46	19.67
A7	10.21	8.61	13.36	7.89
A8	13.27	5.70	12.83	10.94
A9	16.00	10.13	15.03	14.02
A10	17.40	7.05	4.71	15.75
平均	14.07	10.11	11.87	16.36

四季平均增益分别为：春季 14.07、夏季 10.11、秋季 11.87、冬季 16.36。冬季增益

最高，夏季增益最低，冬季平均增益约为夏季的 1.6 倍。冬季水温低，污染物黏性减弱、生物活性降低，附着层较易剥离，清洗效果最佳；夏季水温高、微生物活跃，污垢发展迅速且与膜面结合牢固，维护后恢复量受限。这个季节性规律可为维护的时机选择提供指导：在冬季安排维护可以最大化恢复效果，在夏季可适当增加维护频率或小维护。

综上所述，中维护在瞬时恢复透水率的能力上略优于大维护，但大维护可以短时减缓劣化速度。

5.4 反应透水率变化的影响指标

为量化过滤器透水率的变化规律，从自然衰减、季节影响、维护增益与维护损伤 4 方面设置影响指标，分别为基准劣化率、季节影响算子、维护增益与维护损伤系数。

5.4.1 基准劣化率

基准劣化率 k_0 反映过滤器在无季节干扰下的日均透水率下降速度。将相邻两次维护之间的时间段作为一个独立周期，对各周期内的日均透水率序列进行线性拟合，取斜率的绝对值后平均：

$$k_0 = \frac{1}{M} \sum_{m=1}^M |k_m|, \quad (19)$$

其中 M 为维护间隔的总段数， k_m 为第 m 段内透水率对时间的线性拟合斜率，由最小二乘法求解：

$$\begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_n \end{bmatrix} \begin{bmatrix} a_m \\ k_m \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

5.4.2 季节影响算子

季节影响算子 β_s 刻画不同季节对劣化速率的倍率影响。将各维护间隔按其时间中点的月份归入对应季节，分别计算各季节内分段斜率的平均值，再与基准劣化率取比值：

$$\beta_s = \frac{\bar{k}_s}{k_0}, \quad s \in \{\text{春, 夏, 秋, 冬}\}, \quad (20)$$

其中 $\bar{k}_s$ 为属于季节 s 的所有分段斜率的算术平均值。月份与季节的对应关系为：春季（3-5 月）、夏季（6-8 月）、秋季（9-11 月）、冬季（12-2 月）。

5.4.3 维护增益

维护增益 G 用于量化单次维护操作对过滤器透水率的即时恢复效果。对于每次维护事件，取维护日前后三天内的日均透水率均值之差：

$$G = \frac{1}{n_{\text{after}}} \sum_{i=1}^{n_{\text{after}}} x(t_{\text{after}}^{(i)}) - \frac{1}{n_{\text{before}}} \sum_{i=1}^{n_{\text{before}}} x(t_{\text{before}}^{(i)}), \quad (21)$$

其中 t_{before} 和 t_{after} 分别为维护日前后的邻近时间点， $n_{\text{before}} = n_{\text{after}} = 3$ 。该指标区分中维护与大维护分别统计。

5.4.4 维护损伤系数

维护损伤系数 α 表征维护操作对过滤器造成的不可逆性能退化，即劣化速度的加速比例。首先剔除季节因素对斜率的干扰，得到纯净劣化率：

$$k'_i = \frac{|k_i|}{\beta_{s(i)}},$$

其中 k_i 为第 i 段的有量纲斜率， $\beta_{s(i)}$ 为该段所属季节的季节算子。则损伤系数定义为相邻段纯净劣化率的相对变化率：

$$\alpha = \frac{k'_i - k'_{i-1}}{k'_{i-1}}, \quad (22)$$

$\alpha > 0$ 表示劣化速度加速，即过滤器永久受损； $\alpha \leq 0$ 表示无显著累积损伤。

六、 问题二的模型的建立和求解

根据第一问分析的透水率变化规律，建立基于状态转移的过滤器寿命预测模型。模型综合考虑基准劣化、季节影响、维护增益与维护损伤四个因素，通过逆向回溯算法还原过滤器初始状态，即 2022-2024 年从过滤器启用到附件给出数据的部分，并运用模型正向预测从 2026 年已有数据到寿命终止的部分。

6.1 过滤器寿命预测

首先建立状态转移方程，用以描述过滤器在日常运行中的透水率波动规律，随后给出寿命终止的判定标准，用于确定过滤器寿命终止的具体时刻。

6.1.1 利用 4 个指标探究透水率动态变化

现在我们把在5.4列出的 4 个指标纳入模型，即 (19)-(22) 式：

$$\begin{cases} k_0 = \frac{1}{M} \sum_{m=1}^M |k_m|, \\ \beta_s = \frac{\bar{k}_s}{k_0}, \quad s \in \{\text{春, 夏, 秋, 冬}\}, \\ G = \frac{1}{3} \sum_{i=1}^3 x(t_{\text{after}}^{(i)}) - \frac{1}{3} \sum_{i=1}^3 x(t_{\text{before}}^{(i)}), \\ \alpha = \frac{k'_i - k'_{i-1}}{k'_{i-1}}, \quad k'_i = \frac{|k_i|}{\beta_{s(i)}}. \end{cases}$$

过滤器透水率的每日变化遵循状态转移方程，由劣化损耗与维护恢复共同决定：

$$x(t+1) = x(t) - D(t) + G(t), \quad (23)$$

其中， $x(t)$ 表示第 t 天的透水率， $D(t)$ 为第 t 天的劣化量， $G(t)$ 为第 t 天的维护增益，无维护时 $G(t) = 0$ 。

每日劣化量综合基准劣化、季节影响与维护累积损伤，表达式为：

$$D(t) = k_0 \cdot \beta_{s(t)} \cdot (1 + \alpha_{\text{mid}})^{N_{\text{mid}}(t)} \cdot (1 + \alpha_{\text{big}})^{N_{\text{big}}(t)}, \quad (24)$$

其中： k_0 为基准劣化率， $\beta_{s(t)}$ 为季节影响算子， $N_{\text{mid}}(t)$ 、 $N_{\text{big}}(t)$ 分别为截至第 t 天的中维护、大维护次数； α_{mid} 、 α_{big} 分别为中维护、大维护带来的损伤系数。于是 $(1 + \alpha_{\text{mid}})^{N_{\text{mid}}(t)}$ 与 $(1 + \alpha_{\text{big}})^{N_{\text{big}}(t)}$ 就代表了中维护和大维护的损伤。

维护增益 $G(t)$ 取值决定于当天进行的维护种类或不维护：

$$G(t) = \begin{cases} G_{\text{mid}}, & \text{第 } t \text{ 天进行中维护,} \\ G_{\text{big}}, & \text{第 } t \text{ 天进行大维护,} \\ 0, & \text{无维护操作.} \end{cases} \quad (25)$$

单次维护增益取维护前后各 3 天透水率均值之差：

$$G = \frac{1}{3} \sum_{i=1}^3 x(t_{\text{after}}^{(i)}) - \frac{1}{3} \sum_{i=1}^3 x(t_{\text{before}}^{(i)}). \quad (26)$$

为消除短期波动干扰，采用 365 天滑动平均透水率作为寿命判定的重要指标：

$$\bar{x}_{365}(t) = \frac{1}{365} \sum_{i=t-364}^t x(i). \quad (27)$$

在接下来的6.1.2将其纳入模型作为寿命终止判定的指标。

6.1.2 寿命终止判定

过滤器寿命终止需满足以下任一条件：

$$\text{寿命终止} \iff \underbrace{(\bar{x}_{365}(t) < 37 \wedge \bar{x}_{365}(t+30) < 37)}_{\text{条件一}} \vee \underbrace{(x(t) \leq 0)}_{\text{条件二}},$$

其中 $\bar{x}_{365}(t)$ 为 t 时刻前 365 天的滑动平均透水率。条件一表示年平均透水率低于 37 阈值，且持续 30 天仍无法回升，反映过滤器已不具备稳定过滤能力^[3]；条件二表示透水率完全失效，直接判定寿命终止。

特别地，当过滤器历史记录中存在大维护且尚未在紧急状态下使用时，允许在年均透水率首次下穿 37 时进行一次额外的大维护补偿：

$$x(t_{\text{comp}}) = x(t_{\text{cross}}) + G_{\text{big}}, \quad (28)$$

若补偿后 $\bar{x}_{365}(t_{\text{comp}} + 30) < 37$ 仍成立，则判定寿命终止，否则过滤器继续运行。

6.2 各过滤器的参数运行结果

根据模型，对附件中的数据进行拟合，得到各过滤器的参数如表 7所示：

表 7 各过滤器模型参数确定结果

过滤器	k_0	$\beta_{\text{春}}$	$\beta_{\text{夏}}$	$\beta_{\text{秋}}$	$\beta_{\text{冬}}$	G_{mid}	G_{big}	α_{mid}	α_{big}
A1	0.35413	0.87	0.49	1.38	1.42	16.40	7.72	0.0554	0.1166
A2	0.36594	0.75	0.44	1.37	1.49	12.48	12.82	0.1967	0.0000
A3	0.38178	0.59	0.46	1.57	1.66	12.34	10.28	0.1262	0.0890
A4	0.34367	0.63	0.43	1.48	1.44	16.25	NaN	0.2270	0.0000
A5	0.36588	0.72	0.36	1.49	1.59	13.19	3.96	0.0178	0.0000
A6	0.40957	0.72	0.24	1.50	1.48	18.22	16.66	0.0181	0.1773
A7	0.36094	0.72	0.42	1.58	1.46	10.43	16.59	0.2082	0.0000
A8	0.37760	0.79	0.33	1.55	1.47	10.70	NaN	0.2494	0.0000
A9	0.36145	0.69	0.34	1.26	1.61	13.95	12.89	0.0281	0.3995
A10	0.41143	0.70	0.37	1.54	1.58	11.03	8.12	0.0570	0.7481

6.3 逆向回溯 2022-2024 年的数据

现在利用模型运行出来的各过滤器参数结果逆向回溯还原过滤器在 2022 年 4 月的初始透水率。回溯方程为：

$$x(t-1) = x(t) + k_0 \cdot \beta_{s(t-1)} - G_{\text{hist}}(t-1), \tag{29}$$

其中 G_{hist} 为历史维护日对应的维护增益，通过匹配附件 2 中的维护记录确定。回溯过程逐日递推至投用起始日，得到初始透水率 x_0 。逆向回溯的结果如下：

表 8 各过滤器回溯初始透水率

过滤器	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
回溯初始透水率 x_0	140.81	140.53	140.75	140.24	140.39	140.41	140.48	140.76	140.71	140.52

6.4 寿命预测结果

基于上述模型，采用正向递推算算法预测各过滤器的剩余寿命。预测从 2026 年 4 月实测数据终点开始，按状态转移方程逐日推演，直至满足寿命终止条件。预测过程中维持原有维护规律。

各过滤器的回溯初始透水率及寿命预测结果如下表所示：

表 9 各过滤器寿命预测结果

过滤器	预计寿命终止日期	总运行寿命 (年)
A1	2028-06-20	6.22
A2	2026-12-29	4.74
A3	2027-03-10	4.94
A4	2029-09-12	7.45
A5	2027-09-09	5.44
A6	2028-03-08	5.94
A7	2027-01-19	4.80
A8	2027-01-13	4.79
A9	2028-01-20	5.80
A10	2026-11-26	4.65

由表 9，10 台过滤器的剩余寿命存在显著差异。A4 过滤器寿命最长，达 7.45 年，预计寿命终止日期为 2029-09-12；A10 过滤器寿命最短，仅为 4.65 年，预计寿命终止日期为 2026-11-26。10 台过滤器的平均寿命为 5.48 年。

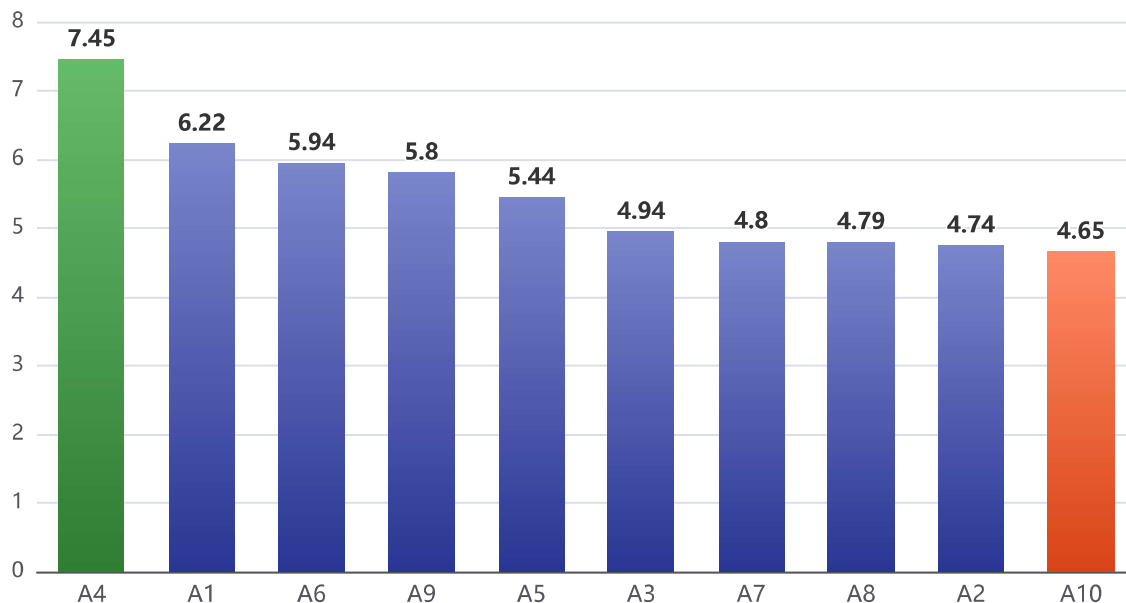


图 10 过滤器 A1-A10 寿命预测结果

各过滤器寿命从高到低排序为：A4、A1、A6、A9、A5、A3、A7、A8、A2、A10。A4 过滤器耐久性最优，A10 过滤器老化速度最快，需优先考虑更换计划。

6.5 透水率预测特征分析

选择过滤器 A9 展示预测结果并分析：

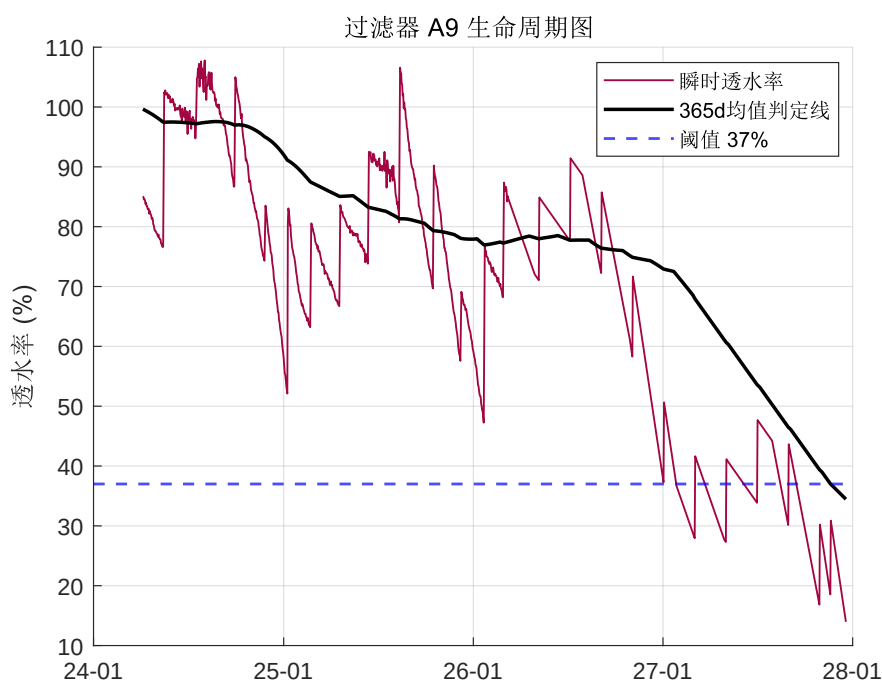


图 11 过滤器 A9 透水率预测曲线

由图 11，预测得到的瞬时透水率呈现典型的锯齿状演化形态，设备每 60 天执行一次中维护，使得透水率出现周期性瞬时回升，展现出显著的周期性波动与阶梯式衰减特征。随着设备服役时间延长与维护次数增加，内部损伤持续累积，中维护的性能恢复效果逐步衰减，透水率的回升幅度不断降低，整体劣化趋势持续加剧。

365 天滑动平均曲线整体呈现平稳下降趋势，同时受季节影响呈现非线性变化特征：夏季劣化速率较慢，曲线走势平缓；秋冬季劣化速度加快，曲线下降趋势更为显著，符合第一问分析的结果。

当滑动平均线首次下穿 37 的寿命阈值时，模型自动触发紧急大维护进行性能补偿；若维护后 30 天内均值曲线仍无法恢复至阈值以上，或瞬时透水率接近失效临界值，则判定设备达到寿命终点。

6.6 模型可靠性检验

为了验证模型的预测可靠性，划分在附件中已知的透水率数据，把前 70% 作为训练集用于模型参数辨识，后 30% 作为测试集用于模型可靠性验证。将测试集预测的数据与原数据集进行对比，计算均方根误差 RMSE 与平均绝对百分比误差 MAPE。

表 10 各过滤器预测误差分析

过滤器	MAPE (%)	RMSE
A1	22.109	21.43
A2	3.7066	2.4178
A3	33.249	27.914
A4	7.0606	8.779
A5	13.338	9.9052
A6	4.8007	5.4529
A7	17.733	15.826
A8	6.331	5.4982
A9	30.606	24.873
A10	9.8331	7.2135

由表 10，在 10 台过滤器中，有 7 台过滤器的 MAPE 低于 18%，其中 A2、A6、A8、A4、A10 等 5 台过滤器的 MAPE 均低于 10%，A2 和 A6 的 MAPE 分别仅为 3.71% 和 4.80%，对应 RMSE 分别为 2.42 和 5.45，预测精度优异。A3 和 A9 的 MAPE 相对较高，分别为 33.25% 和 30.61%，主要是由于该过滤器在验证期内经历了多次大维护，透水率波动剧烈，增加了预测难度。综上，模型对绝大多数过滤器具有可靠的预测能力。

现展示经过训练集参数确认后测试集与原数据集在 2025 年 9 月至 2026 年 4 月的可视化对比：

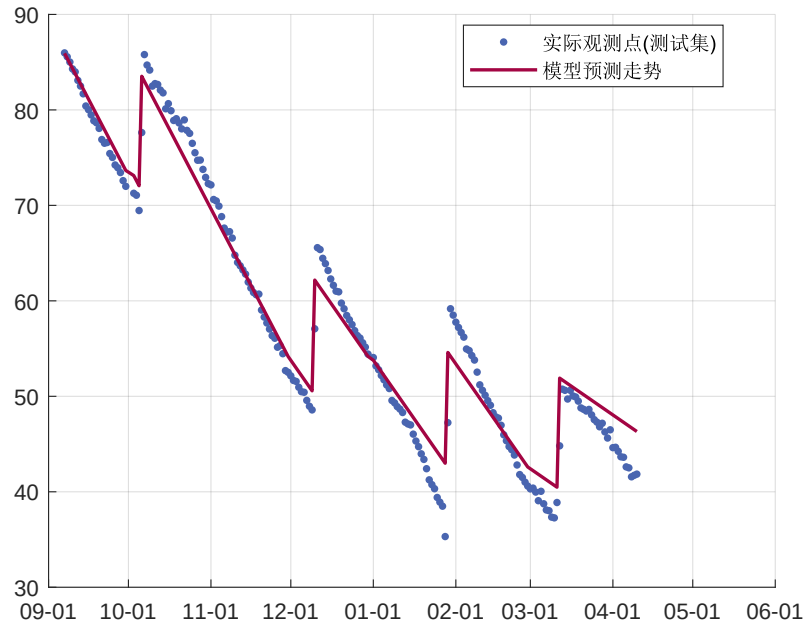


图 12 寿命预测模型预测值与实际值对比

由图 12，模型预测走势与原数据点的整体趋势一致、波动规律相同，表明模型的预测具备可靠性。

七、 问题三的模型的建立和求解

我们首先确定了计算过滤器一年成本的优化函数，根据实际情况确定决策变量，随即运用了前面两问建立的各项指标添加了约束条件，最后使用遗传算法求解，并对比了原来维护规律与我们建立的维护方案的年均成本。

7.1 优化模型构建

为了确定维护方案，在一个过滤器周期内的综合成本最小化，优化模型中优化函数、决策变量、约束条件制定如下。

7.1.1 优化函数

我们以一个过滤器从开始使用到寿命终止作为一个过滤器的生命周期，那么一个过滤器的生命周期的价格就是这个过滤器的价格再加上各种维护成本乘以维护次数，再用一个生命周期折算到一年的价格作为优化函数：

$$\min J(X) = \frac{C_{\text{purchase}} + c_{\text{mid}} \cdot n_{\text{mid}} + c_{\text{big}} \cdot n_{\text{big}}}{L}, \quad (30)$$

其中 $C_{\text{purchase}} = 300$ 万元为过滤器购买价格, $c_{\text{mid}} = 3$ 万元、 $c_{\text{big}} = 12$ 万元分别为中维护与大维护的单次成本, n_{mid} 、 n_{big} 为一个生命周期内两类维护的次数, L 为过滤器寿命(年)。

7.1.2 决策变量

根据现实情况, 当透水率下降较快时, 或距离上次维护时间较长时, 需要进行中维护; 一年进行 0-4 次大维护。根据以上现实需要, 我们制定了三个决策变量:

- **中维护时间间隔阈值 T_{mid} :** 针对距离上次维护时间较长时需进行中维护的要求, 设定当距离上次中维护的天数达到 T_{mid} 时, 进行一次中维护。
- **大维护时间间隔阈值 T_{big} :** 针对过滤器长期运行后性能严重衰减, 需一年进行 0-4 次大维护的要求, 设定当距离上次大维护的天数达到 T_{big} 时, 进行一次大维护。特别地, T_{big} 可以大于一年的时间, 于是一年可能没有大维护。
- **退化速率触发阈值 r_{th} :** 针对透水率下降速率较快时需进行中维护的要求, 设定当近 7 天透水率的平均下降速率超过 r_{th} 时, 即使未达到固定间隔, 也进行中维护。

于是决策向量由三个决策变量组合得到:

$$X = (T_{\text{mid}}, T_{\text{big}}, r_{\text{th}}). \quad (31)$$

7.1.3 约束条件

为确保优化结果符合工程实际, 需对过滤器的运行状态、退化规律及维护触发逻辑施加约束, 具体如下:

1. 确保过滤器始终处于可用状态

过滤器在任意时刻的透水率必须保持正值, 否则意味着过滤器已完全失效, 无法继续使用:

$$P(t) > 0, \quad \forall t. \quad (32)$$

2. 确保长期平均透水率不低于 37

只有 365 天滑动平均值低于 37 时才判定过滤器寿命终止, 以消除短期波动干扰:

$$\bar{P}_{365}(t) = \frac{1}{365} \sum_{i=0}^{364} P(t-i), \quad \bar{P}_{365}(t) \geq 37, \quad \forall t < L. \quad (33)$$

3. 遵循第二问建立的状态转移演化规律

透水率的变化必须严格服从第二问所建立的状态转移方程, 该方程综合了基准

劣化、季节影响、维护损伤累积与维护增益的作用：

$$P(t+1) = P(t) - k_0 \cdot \beta_{s(t)} \cdot (1 + \alpha_{\text{mid}})^{n_{\text{mid}}(t)} \cdot (1 + \alpha_{\text{big}})^{n_{\text{big}}(t)} + G(t). \quad (34)$$

4. 维护执行时机由决策变量控制

大维护按固定时间间隔 T_{big} 执行，中维护在未触发大维护的前提下，由固定间隔 T_{mid} 或下降速率过快两种方式触发：

$$\text{do}_{\text{big}} = (\Delta t_{\text{big}} \geq T_{\text{big}}), \quad (35)$$

$$\text{do}_{\text{mid}} = \neg \text{do}_{\text{big}} \wedge (\Delta t_{\text{mid}} \geq T_{\text{mid}} \vee \text{rate}(t) \leq -r_{\text{th}}), \quad (36)$$

其中 $\text{rate}(t) = \frac{P(t) - P(t-7)}{7}$ 为过去 7 天的平均下降速率， Δt_{mid} 、 Δt_{big} 分别为距上次中维护、大维护的时间间隔。

综上，优化模型的完整约束条件为：

$$\left\{ \begin{array}{l} P(t) > 0, \quad \forall t \\ \bar{P}_{365}(t) = \frac{1}{365} \sum_{i=0}^{364} P(t-i), \quad \bar{P}_{365}(t) \geq 37, \quad \forall t < L \\ P(t+1) = P(t) - k_0 \cdot \beta_{s(t)} \cdot (1 + \alpha_{\text{mid}})^{n_{\text{mid}}(t)} \cdot (1 + \alpha_{\text{big}})^{n_{\text{big}}(t)} + G(t) \\ \text{do}_{\text{big}} = (\Delta t_{\text{big}} \geq T_{\text{big}}) \\ \text{do}_{\text{mid}} = \neg \text{do}_{\text{big}} \wedge (\Delta t_{\text{mid}} \geq T_{\text{mid}} \vee \text{rate}(t) \leq -r_{\text{th}}) \\ \text{rate}(t) = \frac{P(t) - P(t-7)}{7} \end{array} \right.$$

7.2 利用遗传算法制定维护策略

遗传算法是一类借鉴生物界自然选择和自然遗传机制的随机搜索算法。该算法模拟自然选择和自然遗传过程中发生的繁殖、交叉和基因突变现象，在每次迭代中保留一组候选解，并按适应度指标从解群中选取较优个体，利用遗传算子对这些个体进行组合产生新一代候选解群，重复此过程直至满足收敛指标^[4]。与传统的启发式优化搜索算法相比，遗传算法的主要本质特征在于群体搜索策略和简单的遗传算子，群体搜索使算法能够突破领域搜索的限制，实现整个解空间上的分布式信息采集和探索；遗传算子仅利用适应值度量作为运算指标进行随机操作，降低了一般启发式算法在搜索过程中对人工交互的依赖。

基于该优化问题的以下三个特点，选择遗传算法求解：

- 动态系统约束是非线性的

由状态转移方程 $P(t+1) = P(t) - k_0 \cdot \beta_{s(t)} \cdot (1 + \alpha_{\text{mid}})^{n_{\text{mid}}(t)} \cdot (1 + \alpha_{\text{big}})^{n_{\text{big}}(t)} + G(t)$ 可知，透水率每天的变化量不仅取决于当前值，还与维护次数 n_{mid} 、 n_{big} 呈指数关系，那么维护次数越多，后续每天的劣化速度就越快。同时，季节因子 $\beta_{s(t)}$ 会在不同月时跳变，比如夏季约为 0.4，秋季约为 1.5，导致劣化速度相差近 4 倍。这些非线性、离散跳变和动态演化共同构成了复杂的约束体系。

- 维护行为会导致状态跳变

维护增益 $G(t)$ 的触发规则为

$$\text{do}_{\text{mid}} = \neg \text{do}_{\text{big}} \wedge (\Delta t_{\text{mid}} \geq T_{\text{mid}} \vee \text{rate}(t) \leq -r_{\text{th}}),$$

当条件满足时，透水率会几乎突然增加， G_{mid} 或 G_{big} 会发生跳变，不是连续的。

- 目标函数不可解析表达

年均成本 $J = (300 + 3n_{\text{mid}} + 12n_{\text{big}})/L$ 中的 n_{mid} 、 n_{big} 和 L ，无法写成关于 T_{mid} 、 T_{big} 、 r_{th} 的公式。例如， L 取决于逐日仿真中透水率何时跌破阈值、何时触发维护、衰减速率的累积效应等，这些过程包含大量条件判断和离散事件，无法用一个公式直接计算。

遗传算法不要求目标函数可导或可解析表达，仅需能够计算每个个体的适应度值，非常契合上述特点，因此采用遗传算法对该维护策略优化问题进行求解。

遗传算法参数设置如下：种群规模为 30，最大进化代数 25，交叉概率 0.7，变异概率 0.05。决策变量搜索范围为： $T_{\text{mid}} \in [40, 120]$ 天， $T_{\text{big}} \in [150, 400]$ 天， $r_{\text{th}} \in [0.05, 1.5]$ 透水率/天。算法流程如下：

```

初始化种群  $\{X_1, X_2, \dots, X_{N_{\text{pop}}}\}$ 
for  $gen = 1$  to  $N_{\text{gen}}$  :
    for each 个体  $X_i$  :
        运行寿命仿真，得  $L_i, n_{\text{mid}}^i, n_{\text{big}}^i$ 
        计算适应度  $J_i = (300 + 3n_{\text{mid}}^i + 12n_{\text{big}}^i)/L_i$ 
    end
    选择、交叉、变异生成新一代种群
end
输出使  $J$  最小的  $X^*$ 

```

采用 MATLAB 内置遗传算法函数 **ga** 求解：

```

1 bestX = ga(@(X) fitness_func(X, value_series, start_date, ...
2 k0, S_vals, gm, gb, alpha_m, alpha_b, m_types, t_born), ...
3 3, [], [], [], [], lb, ub, [], options);

```

其中 `fitness_func` 为目标函数，返回给定决策变量 $X = (T_{\text{mid}}, T_{\text{big}}, r_{\text{th}})$ 下的年均成本；`lb`、`ub` 为决策变量下界与上界；`options` 通过 `optimoptions('ga', ...)` 设置种群规模为 30、最大进化代数为 25 等参数。

7.3 遗传算法的求解结果

采用遗传算法对 10 台过滤器分别进行优化求解，得到的最优维护策略及对应寿命、年均成本如表 11 所示：

表 11 遗传算法优化结果

过滤器	L (年)	J (万元)	T_{mid} (天)	T_{big} (天)	r_{th}
A1	5.51	60.42	63	356	1.34
A2	4.53	66.87	115	273	1.33
A3	4.68	64.80	120	277	1.20
A4	4.88	62.65	118	329	1.33
A5	5.81	60.44	45	384	1.25
A6	5.72	59.24	62	379	1.15
A7	4.60	65.88	114	399	1.15
A8	4.58	66.11	119	274	0.93
A9	5.57	60.28	60	391	1.47
A10	4.55	66.59	115	310	1.14

其中 L 为过滤器寿命 (年)， J 为年均成本 (万元/年)， T_{mid} 为中维护间隔阈值 (天)， T_{big} 为大维护间隔阈值 (天)， r_{th} 为退化速率触发阈值。经遗传算法优化后，工厂的污水净化年均成本，即 10 台过滤器的年均总成本为 635.5 万元。

为验证优化效果，将优化策略与第二问的原始固定维护策略进行对比，原始策略与优化策略的各过滤器的年均成本如表 12 所示：

表 12 优化策略与原始策略对比

过滤器	优化后寿命	优化后年均成本	原始寿命	原始年均成本	成本波动
A1	5.52	60.36	6.22	65.11	-7.30%
A2	4.53	66.87	4.74	81.56	-18.01%
A3	4.72	64.87	4.94	75.32	-13.88%
A4	4.88	62.65	7.45	53.56	+16.97%
A5	5.68	60.70	5.44	70.59	-14.01%
A6	6.02	59.30	5.94	67.22	-11.78%
A7	4.60	65.88	4.80	76.84	-14.26%
A8	4.58	66.11	4.79	73.34	-9.85%
A9	5.50	62.21	5.80	68.74	-9.49%
A10	4.55	66.59	4.65	80.57	-17.35%
平均	5.06	63.55	5.58	71.38	-11.04%

由表 12, 经遗传算法优化后, 10 台过滤器中有 9 台的年均成本较原始策略显著下降, 平均降幅达 11.04%。其中 A2、A10、A7、A5 降幅最为明显, 分别下降 18.01%、17.35%、14.26% 和 14.01%, A3 和 A6 降幅分别为 13.88% 和 11.78%, 验证了遗传算法在搜索最优维护策略方面的有效性。A8 和 A9 降幅分别为 9.85% 和 9.49%, A1 降幅为 7.30%。A4 过滤器成本上升 16.97%, 主要由于其原始数据中缺乏大维护记录, 导致参数辨识存在偏差, 优化效果不显著。总体而言, 优化策略在保证过滤器可靠运行的前提下, 能够有效降低年均成本, 验证了模型的有效性与实用性。

接下来选择过滤器 A5 在优化策略下的透水率预测曲线进行展示:



图 13 过滤器 A5 在新策略下的透水率预测曲线

八、 问题四的模型的建立和求解

需要改变过滤器购买价格与维护成本来验证第三问的维护策略的稳健性, 即对过滤器购买价格与维护成本进行敏感性分析。在题目给的三个价格参数的基础上, 设置四类波动情景: 购买价格单独波动、维护成本单独波动、全部成本同向波动、购买价格与维护成本反向波动, 波动幅度为 $\pm 10\%$ 和 $\pm 15\%$ 。

为了衡量第三问建立的策略在新成本下的表现, 设损失率 δ 为:

$$\delta = \frac{A_{\text{fixed}} - A_{\text{opt}}}{A_{\text{opt}}} \times 100\%, \quad (37)$$

其中, A_{fixed} 为保持原维护策略不变、仅替换成本单价后计算得到的年均成本; A_{opt} 为在新成本参数下重新优化得到的最优年均成本。 δ 为正表示原策略优于重新优化策略, 为负表示原策略劣于重新优化策略。 $|\delta| \leq 3\%$ 代表原方案完全可行, $3\% < |\delta| \leq 5\%$ 代表模型轻微敏感。

选择展示过滤器 A1 在所有参数波动情景下的敏感性分析结果：

表 13 过滤器 A1 成本敏感性分析结果

情景	A_{fixed} (万元)	A_{opt} (万元)	δ (%)
购买价格 +10%	65.86	65.80	0.10
购买价格 -10%	54.98	55.03	-0.09
购买价格 +15%	68.59	69.00	-0.60
购买价格 -15%	52.26	52.20	0.10
维护成本 +10%	61.02	60.99	0.05
维护成本 -10%	59.82	60.15	-0.55
维护成本 +15%	61.32	61.27	0.08
维护成本 -15%	59.52	59.49	0.05
全部成本 +10%	66.46	66.63	-0.24
全部成本 -10%	54.38	54.38	0.00
全部成本 +15%	69.48	71.89	-3.34
全部成本 -15%	51.36	51.34	0.04
购买价格 +10% 维护成本 -10%	65.27	66.11	-1.28
购买价格 -10% 维护成本 +10%	55.58	55.72	-0.25
购买价格 +15% 维护成本 -15%	67.69	67.65	0.06
购买价格 -15% 维护成本 +15%	53.15	53.15	0.00

由表 13，在所有测试情景下，损失率 δ 的绝对值均不超过 3.34%，其中绝大多数情景的 $|\delta|$ 小于 2%。仅当全部成本上涨 15% 时， $\delta = -3.34\%$ ，略超过 3% 阈值，但仍处于可接受范围内。综上，第三问的最优维护策略在购买价格和维护成本发生 $\pm 15\%$ 以内的变化时体现出良好的稳健性。

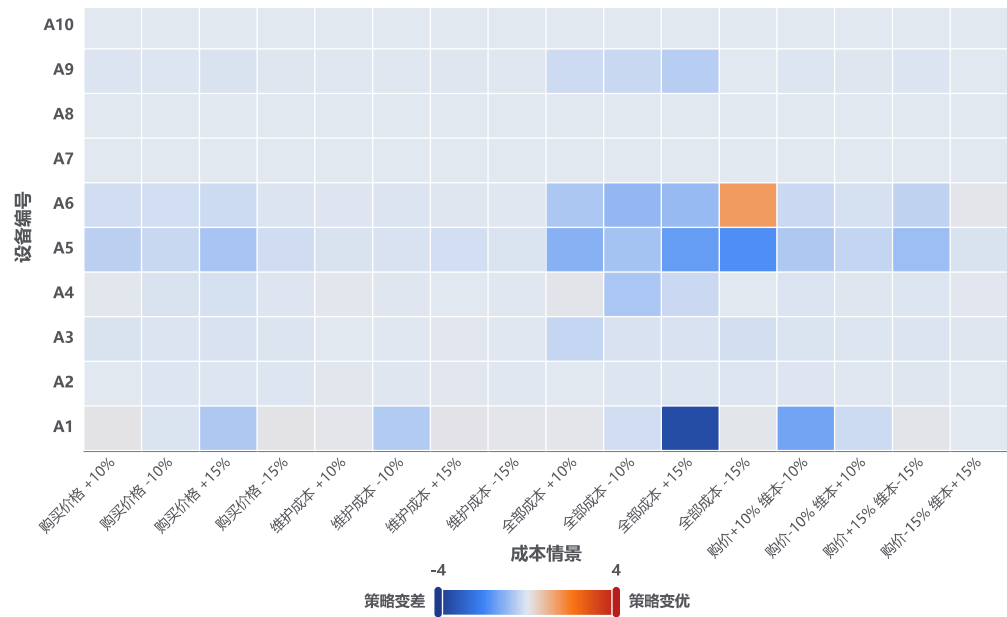


图 14 过滤器维护策略损失率热力图

由图 14，绝大多数 δ 值落在 -1% 至 1% 之间，A7、A8、A10 在所有情景下 δ 均为零，A5 和 A6 的 δ 绝对值略高但不超过 1.70% ，所有情景最大的 $|\delta|$ 为 A1 在全部成本上涨 15% 时的 -3.34% 。所有 $|\delta|$ 均控制在 3.34% 以内，验证了最优维护策略在成本波动下的良好稳健性。

对所有设备在全部成本波动情景下的损失率进行展示：

表 14 全部过滤器在全部成本波动下的损失率（%）

过滤器	全部成本 +10%	全部成本 -10%	全部成本 +15%	全部成本 -15%
A1	0.05	-0.20	0.11	0.06
A2	0.00	-0.06	-0.06	-0.06
A3	-0.34	-0.11	-0.11	-0.17
A4	0.06	-0.64	-0.28	0.00
A5	-1.03	-0.71	-1.44	-1.70
A6	-0.62	-0.92	-0.86	1.26
A7	0.00	0.00	0.00	0.00
A8	0.00	0.00	0.00	0.00
A9	-0.25	-0.30	-0.49	0.00
A10	0.00	0.00	0.00	0.00

由表 14，在全部成本同向波动 $\pm 10\%$ 和 $\pm 15\%$ 的情景下，所有过滤器的损失率绝对值均未超过 1.70% ，远低于 3% 的可行性阈值。其中 A7、A8、A10 三台设备的损失率在所有情景下均为 0，表明其原最优策略在成本波动后依然处于最优解附近。A2 的损失率绝对值最大为 0.06% ，A1 和 A4 的损失率绝对值均在 0.64% 以内，A3 和 A9 的损失率绝对值均在 0.49% 以内，均表现稳定。A5 和 A6 的损失率相对略高，A5 在全部成本下降 15% 时损失率为 -1.70% ，A6 在同一情景下损失率为 1.26% ，但绝对值仍远低于 3% 的阈值。综合全部设备的分析结果，第三问所求得的最优维护策略在成本发生 $\pm 10\% \sim \pm 15\%$ 波动时具有良好的普适性和稳健性，无需因市场价格波动而调整维护方案。

九、 模型的评价

9.1 模型的优点

- **数据预处理充分：**采用改进的自适应 Hampel 滤波器对原始数据进行清洗，有效剔除了异常波动并保留维护引发的真实阶跃信号。
- **物理意义清晰：**模型综合考虑了具有明确物理含义的自然衰减、季节影响、维护增益与损伤四项指标，避免了单一因素分析的片面性与黑箱模型的不可解释性。
- **模型可解释性强：**透水率状态转移方程基于实际物理过程构建，各参数均可从数据中辨识，避免了黑箱模型难以解释的弊端。

- **优化方法适用性强：**以年均成本最小化为目标函数，将维护间隔与速率阈值作为决策变量，采用遗传算法全局寻优，有效处理了离散决策变量与非线性目标函数的组合优化问题，对各设备均能稳定获得合理方案，复现性良好。
- **模型泛化性良好：**模型可推广应用于不同型号、不同工况的过滤设备。只需根据具体设备的透水率监测数据重新辨识衰减系数、季节算子、损伤系数等核心参数，即可适配不同设备的性能特征与维护需求，具有较强的可扩展性。

9.2 模型的缺点

- **未考虑小维护的影响：**由于小维护成本可忽略且单次效果微弱，没有将其纳入模型。对于小维护效果显著的场景，模型可能需要进一步扩展。
- **季节影响采用离散跳变：**季节算子按月离散取值，未能刻画季节交替过程中的连续过渡特征，可能引入一定近似误差。

9.3 模型的改进方向

- **精细化季节影响建模：**当前季节算子按月离散跳变，未来可采用连续函数拟合季节波动，更精确地刻画季节交替过程中的过渡特征。
- **引入参数不确定性量化：**透水率检测存在测量噪声，参数辨识和寿命预测结果存在不确定性，未来可引入贝叶斯方法等概率模型对不确定性进行量化。
- **考虑外部环境的随机波动：**后续可引入随机微分方程或蒙特卡洛仿真，进一步描述污水水质、操作条件、气候条件变化等随机波动对模型的影响，提升对不确定因素的量化能力与决策的稳健性。

9.4 模型的推广

- **适用其他工业设备的维护优化：**模型可推广至其他存在性能退化且需要定期维护的工业设备，如空气过滤器、油液过滤器、膜分离设备等。
- **可包容更多影响因素：**模型框架可扩展纳入更多影响因素，如操作压力、流量、温度等实时运行参数，只需要将参数纳入状态转移方程即可。
- **可扩展至多目标优化方案：**当前以年均成本最小为单目标，同时也具有可扩展为多目标优化，只需要添加对应参数至优化函数，并添加对应决策变量及限制条件即可。

参考文献

[1] 谢宇轩, 范琳琳, 郭鑫, 等. 基于 Hampel 滤波的海洋磁测数据异常值检测方法研究 [J]. 海洋学报, 2025, 47(04): 53-64.

[2] 张闻胜, 王银堂. 时间序列分解模型在水文要素中长期预报中的应用 [J]. 水文, 2001, (01): 21-24.

[3] CJJ 60-2011, 城镇污水处理厂运行、维护及安全技术规程 [S]. 北京: 中国建筑工业出版社, 2011.

[4] 边霞, 米良. 遗传算法理论及其应用研究进展 [J]. 计算机应用研究, 2010, 27(07): 2425-2429+2434.

附录 A 支撑材料的文件列表

文件名	功能描述
附件 1 整合数据.xlsx	用于读入代码的数据表
附件 2 整合数据.xlsx	用于读入代码的数据表
Q1_Hampel_All.m	第一问进行异常值处理的代码
Q1_STL.m	第一问进行时间序列分解的代码
Q1_Parameter.m	第一问下降趋势规律分析代码
Q2_Finalist.m	第二问参数确定与寿命预测代码
Q2_Reliability.m	第二问模型可靠性检验代码
Q3Q4.m	第三、四问利用遗传算法进行优化模型求解以及参数扰动检验代码

附录 B 代码

Q1_Hampel_All.m

```
1 %% 第一问数据清洗处理
2 clear; clc; close all;
3 % 数据导入
4 filename = '附件1整合数据.xlsx'; % 数据文件
5 deviceNum = 10; % 设备数量
6 outputFolder = '设备图表输出'; % 图片保存文件夹
7 if ~exist(outputFolder, 'dir')
8     mkdir(outputFolder);
9 end
10 %跨设备数据存储
11 allTime = cell(deviceNum,1);
12 allPerFinal = cell(deviceNum,1);
13 allMaintDates = cell(deviceNum,1);
14 allMaintTypes = cell(deviceNum,1);
15 deviceNames = cell(deviceNum,1);
16
17 % 利用Hampel滤波器进行异常值处理
18 function is_out = adaptive_hampel(per_vec, time_vec, is_exempt, ...
19     minPts, maxHrs, thr)
20     is_out = false(size(per_vec));
21     maxDur = hours(maxHrs);
22     for i = 1:length(per_vec)
23         if isnan(per_vec(i)) || is_exempt(i), continue; end
```

```

24     t_i = time_vec(i);
25     % 向前收集点
26     leftIdx = [];
27     for j = i-1 : -1 : 1
28         if t_i - time_vec(j) > maxDur, break; end
29         if ~isnan(per_vec(j)) && ~is_exempt(j)
30             leftIdx = [leftIdx, j]; %#ok<AGROW>
31             if length(leftIdx) >= minPts, break; end
32         end
33     end
34     % 向后收集点
35     rightIdx = [];
36     for j = i+1 : length(per_vec)
37         if time_vec(j) - t_i > maxDur, break; end
38         if ~isnan(per_vec(j)) && ~is_exempt(j)
39             rightIdx = [rightIdx, j]; %#ok<AGROW>
40             if length(rightIdx) >= minPts, break; end
41         end
42     end
43     if length(leftIdx) < minPts || length(rightIdx) < minPts
44         continue;
45     end
46     windowPer = per_vec([leftIdx, rightIdx]);
47     med = median(windowPer);
48     mad_val = median(abs(windowPer - med));
49     if mad_val == 0
50         score = inf * (per_vec(i) ~= med);
51     else
52         score = abs(per_vec(i) - med) / mad_val;
53     end
54     if score > thr, is_out(i) = true; end
55 end
56 end
57
58 for dev = 1:deviceNum
59     sheetName = ['A_', num2str(dev)];
60     % 数据读取
61     opts = detectImportOptions(filename, 'Sheet', sheetName);
62     opts = setvaropts(opts, 'time', 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
63     data = readtable(filename, opts);
64
65     % 维护日期 (F列) 和维护类型 (G列)
66     if ismember('日期', data.Properties.VariableNames)
67         maintDateCol = data('日期');
68     else
69         maintDateCol = data(:, 6);
70     end
71     if ismember('维护类型', data.Properties.VariableNames)
72         maintTypeCol = data('维护类型');
73     else
74         maintTypeCol = data(:, 7);
75     end
76     validMaint = ~isnan(maintDateCol);
77     maintDates = maintDateCol(validMaint);
78     maintTypes = maintTypeCol(validMaint);
79     maintDates = dateshift(maintDates, 'start', 'day');
80
81     % 透水率时间和值, 排序
82     time = data.time;
83     per = data.per;
84     [time, idx] = sort(time);
85     per = per(idx);
86     % 维护标记
87     maintWindow = hours(24);
88     is_maint_day = false(size(time));
89     is_maint_24h = false(size(time));
90     for i = 1:length(time)
91         thisDay = dateshift(time(i), 'start', 'day');
92         if ismember(thisDay, maintDates)
93             is_maint_day(i) = true;
94             is_maint_24h(i) = true;
95         else
96             for d = 1:length(maintDates)
97                 if time(i) > maintDates(d) && ...
98                     time(i) <= (maintDates(d) + days(1) + maintWindow)
99                     is_maint_24h(i) = true;
100                     break;
101                 end
102             end
103         end
104     end
105
106     % 初始处理输出

```

```

107 fprintf('总点数: %d, 维护日: %d, 维护日+24h: %d, 原始缺失: %d\n', ...
108         length(per), sum(is_maint_day), sum(is_maint_24h), sum(isnan(per)));
109
110 % 第一轮异常检测
111 minPts1 = 5; maxHrs1 = 72; thr1 = 3;
112 is_outlier1 = adaptive_hampel(per, time, is_maint_24h, minPts1, maxHrs1, thr1);
113 fprintf('第一轮异常点: %d\n', sum(is_outlier1));
114 per_temp = per;
115 per_temp(is_outlier1) = NaN;
116 per_filled1 = fillmissing(per_temp, 'linear', 'SamplePoints', time);
117 per_filled1(isnan(per)) = NaN;
118
119 % 第二轮检测 (维护后24h窗口)
120 minPts2 = 5; maxHrs2 = 72; thr2 = 3;
121 is_outlier2 = false(size(per));
122 targetIdx = find(is_maint_24h & ~is_maint_day);
123 for idx = 1:length(targetIdx)
124     i = targetIdx(idx);
125     if isnan(per_filled1(i)), continue; end
126     t_i = time(i);
127     leftIdx = []; rightIdx = [];
128     for j = i-1 : -1 : 1
129         if t_i - time(j) > hours(maxHrs2), break; end
130         if ~isnan(per_filled1(j)) && ~is_maint_day(j)
131             leftIdx = [leftIdx, j]; %#ok<AGROW>
132             if length(leftIdx) >= minPts2, break; end
133         end
134     end
135     for j = i+1 : length(per_filled1)
136         if time(j) - t_i > hours(maxHrs2), break; end
137         if ~isnan(per_filled1(j)) && ~is_maint_day(j)
138             rightIdx = [rightIdx, j];
139             if length(rightIdx) >= minPts2, break; end
140         end
141     end
142     if length(leftIdx) < minPts2 || length(rightIdx) < minPts2
143         continue;
144     end
145     windowPer = per_filled1([leftIdx, rightIdx]);
146     med = median(windowPer);
147     mad_val = median(abs(windowPer - med));
148     if mad_val == 0
149         score = inf * (per_filled1(i) ~= med);
150     else
151         score = abs(per_filled1(i) - med) / mad_val;
152     end
153     if score > thr2, is_outlier2(i) = true; end
154 end
155 fprintf('第二轮异常点: %d\n', sum(is_outlier2));
156
157 % 异常点整合
158 is_outlier_final = is_outlier1 | is_outlier2;
159 fprintf('总异常点: %d\n', sum(is_outlier_final));
160
161 per_clean = per;
162 per_clean(is_outlier_final) = NaN;
163 maint_nan = isnan(per) & is_maint_24h;
164 per_final = fillmissing(per_clean, 'linear', 'SamplePoints', time);
165 per_final(maint_nan) = NaN;
166
167 % 保存结果到excel
168 time_str = cellstr(datestr(time, 'yyyy-mm-dd HH:MM:SS'));
169 result = table(time_str, per, is_maint_24h, is_outlier1, is_outlier2, is_outlier_final,
170               per_final, ...
171               'VariableNames', {'Time', 'begin_per', 'maintain_24h', 'outlier1', 'outlier2', 'outlier',
172                                'final_per'});
173 excelName = ['设备', sheetName, '_清洗结果_含缺失填补.xlsx'];
174 writetable(result, excelName);
175 fprintf('清洗结果已保存为 %s\n', excelName);
176
177 % 单设备效果图
178 % 图一: 原始数据散点图
179 figRow = figure('Name', [sheetName, ' 原始数据散点图'], 'Position', [100 100 1000 500]);
180 scatter(time, per, 10, 'filled', 'MarkerFaceColor', '#387b85', 'MarkerEdgeColor', 'none');
181 xlabel('时间'); ylabel('透水率 (%)');
182 title([sheetName, ' 原始数据散点图']);
183 grid on;
184 for k = 1:length(maintDates)
185     xline(maintDates(k), '--', 'color', '#84c3b7', 'LineWidth', 1.5);
186 end
187 saveas(figRow, fullfile(outputFolder, [sheetName, '_原始散点图.png']));
188 close(figRow);

```

```

188 % 图二：清洗后数据散点图
189 figClean = figure('Name', [sheetName, ' 清洗后数据散点图'], 'Position', [100 100 1000 500]);
190 cleanIdx = ~isnan(per_final);
191 scatter(time(cleanIdx), per_final(cleanIdx), 10, 'filled', 'MarkerFaceColor', '#387b85', '
    MarkerEdgeColor', 'none');
192 xlabel('时间'); ylabel('透水率 (%)');
193 title([sheetName, ' 清洗后数据散点图']);
194 grid on;
195 for k = 1:length(maintDates)
196     xline(maintDates(k), '--', 'color', '#84c3b7', 'LineWidth', 1.5);
197 end
198 saveas(figClean, fullfile(outputFolder, [sheetName, '_清洗后散点图.png']));
199 close(figClean);
200
201 % 存储跨设备分析所需数据
202 allTime{dev} = time;
203 allPerFinal{dev} = per_final;
204 allMaintDates{dev} = maintDates;
205 allMaintTypes{dev} = maintTypes;
206 deviceNames{dev} = sheetName;
207 end
208 disp('全部设备处理完成!');
209 %%跨设备箱线图：不同维护类型恢复幅度对比
210 allMid = [];
211 allBig = [];
212 for dev = 1:deviceNum
213     time = allTime{dev};
214     per_final = allPerFinal{dev};
215     maintDates = allMaintDates{dev};
216     maintTypes = allMaintTypes{dev};
217
218     for k = 1:length(maintDates)
219         idxBefore = find(time < maintDates(k) & ~isnan(per_final), 1, 'last');
220         idxAfter = find(time > maintDates(k) & ~isnan(per_final), 1, 'first');
221         if ~isempty(idxBefore) && ~isempty(idxAfter)
222             amp = per_final(idxAfter) - per_final(idxBefore);
223             if contains(maintTypes(k), '大')
224                 allBig = [allBig; amp];
225             else
226                 allMid = [allMid; amp];
227             end
228         end
229     end
230 end
231 dataPlot = [allMid; allBig];
232 grp = [repmat({'中维护'}, length(allMid), 1); repmat({'大维护'}, length(allBig), 1)];
233 if ~isempty(dataPlot)
234     figure('Name', '跨设备恢复幅度箱线图', 'Position', [100 100 800 500]);
235     myColors = [hex2dec('80')/255, hex2dec('cb')/255, hex2dec('a4')/255; % #80cba4
236                 hex2dec('49')/255, hex2dec('65')/255, hex2dec('b0')/255]; % #4965b0
237     h=boxplot(dataPlot, grp, 'Colors', myColors);
238     set(h, 'LineWidth', 1.5);
239     hold on;
240     if ~isempty(allMid)
241         scatter(ones(size(allMid)) + 0.1*(rand(size(allMid))-0.5), allMid, ...
242                 20, myColors(1,:), 'filled');
243     end
244     if ~isempty(allBig)
245         scatter(2*ones(size(allBig)) + 0.1*(rand(size(allBig))-0.5), allBig, ...
246                 20, myColors(2,:), 'filled');
247     end
248     ylabel('恢复幅度', 'FontSize', 14);
249     title('不同维护类型恢复幅度对比', 'FontSize', 14);
250     set(gca, 'FontSize', 14);
251     grid on;
252     saveas(gcf, fullfile(outputFolder, 'Q1_恢复幅度箱线图.png'));
253     fprintf('跨设备箱线图已保存至 %s\n', outputFolder);
254 end

```

code/Q1_Hampel_All.m

Q1_STL.m

```

1 %第一问周期性规律、下降趋势分析
2 clear; clc; close all;
3 %数据读取
4 maint_file = '附件2整合数据.xlsx';
5 if ~isfile(maint_file)
6     error('请将附件2整合数据.xlsx放在当前目录');
7 end
8 protect = detectImportOptions(maint_file);

```

```

9 protect = setvartype(protect, {'ID'}, 'string');
10 protect = setvartype(protect, {'DATE'}, 'string');
11 maint_data = readtable(maint_file, protect);
12 maint_data.DATE = datetime(maint_data.DATE, 'InputFormat', 'yyyy-MM-dd');
13 type_num = zeros(height(maint_data),1);
14 type_num(contains(maint_data.LEIBIE,'中')) = 1;
15 type_num(contains(maint_data.LEIBIE,'大')) = 2;
16 maint_data.TypeNum = type_num;
17 device_ids = unique(maint_data.ID);
18 fprintf('共 %d 台设备待分析\n\n', length(device_ids));
19 all_summaries = {};
20 all_seg_info = table();
21 mkdir('Figures');
22 selected_devs = {'A1','A2','A3','A4','A5'};
23 season_cell = cell(1,5); % 每个cell存放 [datenum, seasonal]
24 season_dev_names = strings(1,5);
25 season_amp = zeros(1,5); % 存储对应设备的A_s
26 %逐台设备进行分析
27 for d = 1:length(device_ids)
28     dev = device_ids(d);
29     fprintf('==== 设备 %s =====\n', dev);
30     if ~isempty(regexp(dev, '[A-Za-z]\d', 'once'))
31         dev_mod = regexp(dev, '([A-Za-z])(\d)', '$1_$2');
32     else
33         dev_mod = dev;
34     end
35     data_file = sprintf('设备%s_清洗结果_含缺失填补.xlsx', dev_mod);
36     if ~isfile(data_file)
37         warning('文件 %s 不存在, 跳过', data_file);
38         continue;
39     end
40     opts_d = detectImportOptions(data_file, 'Sheet', 'Sheet1');
41     opts_d = setvartype(opts_d, {'Time'}, 'string');
42     data = readtable(data_file, opts_d);
43     if any(strcmp(data.Properties.VariableNames, 'final_per'))
44         col_perm = 'final_per';
45     elseif any(strcmp(data.Properties.VariableNames, '清洗后透水率'))
46         col_perm = '清洗后透水率';
47     else
48         error('未找到透水率列, 请修改代码中的列名');
49     end
50     data = rmmissing(data, 'DataVariables', {'Time', col_perm});
51     data.Time = datetime(data.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
52     data = sortrows(data, 'Time');
53     time = data.Time;
54     permeability = data.(col_perm);
55 %设备维护记录导入
56 idx_mt = maint_data.ID == dev;
57 if sum(idx_mt) == 0
58     warning('设备 %s 无维护记录, 跳过', dev);
59     continue;
60 end
61 m_dates = maint_data.DATE(idx_mt);
62 m_types = maint_data.TypeNum(idx_mt);
63 [m_dates, sort_idx] = sort(m_dates);
64 m_types = m_types(sort_idx);
65 %构建日均透水率, 便于分析
66 [dates_u, ~, idx_u] = unique(dateshift(time, 'start', 'day'));
67 daily_per = accumarray(idx_u, permeability, [], @mean);
68 if any(isnan(daily_per))
69     daily_per = fillmissing(daily_per, 'linear');
70 end
71 t_start = dates_u(1);
72 t_end = dates_u(end);
73 daily_dates = (t_start:days(1):t_end)';
74 daily_per_full = interp1(dates_u, daily_per, daily_dates, 'linear', 'extrap');
75 %利用STL进行周期性规律分析
76 try
77     [trend, seasonal, residual] = trenddecomp(daily_per_full, 'stl', 365);
78 catch
79     warning('trenddecomp 不可用, 使用经典季节分解');
80     [trend, seasonal, residual] = classical_seasonal_decomp(daily_per_full, 365);
81 end
82 A_s = max(seasonal) - min(seasonal);
83 if any(strcmp(dev, selected_devs))
84     idx = find(strcmp(selected_devs, dev));
85     season_cell{idx} = [datenum(daily_dates), seasonal];
86     season_dev_names(idx) = dev;
87     season_amp(idx) = A_s;
88 end
89 % STL 分解图
90 figure('Visible','off');
91 figure('Visible','off');

```

```

92 subplot(4,1,1); plot(daily_dates, daily_per_full, 'color','#f57c6e'); title(sprintf('设备%s
    原始日均序列', dev)); grid on;
93 subplot(4,1,2); plot(daily_dates, trend, 'color','#71b8ed'); title('趋势项'); grid on;
94 subplot(4,1,3); plot(daily_dates, seasonal, 'color','#f57c6e'); title(sprintf('季节项 (振幅
    =%.2f)', A_s)); grid on;
95 subplot(4,1,4); plot(daily_dates, residual, 'color','#b8aeaa'); title('残差项'); grid on;
96 saveas(gcf, fullfile('Figures', sprintf('%s_STL分解.png', dev)));
97 close(gcf);
98 % FFT 频谱 (去趋势后)
99 y_det = detrend(daily_per_full);
100 n_y = length(y_det);
101 Y = fft(y_det);
102 power = abs(Y).^2 / n_y;
103 half = floor(n_y/2);
104 f_axis = (0:half-1)' * (1/n_y);
105 pow_plot = power(1:half);
106 [~, i_max] = max(pow_plot(2:end));
107 T_max = 1 / f_axis(i_max+1);
108 figure('Visible','off');
109 plot(f_axis*365, pow_plot); xlabel('频率 (周期/年)'); ylabel('功率');
110 title(sprintf('设备%s FFT 功率谱 (主周期≈%.0f天)', dev, T_max)); grid on;
111 saveas(gcf, fullfile('Figures', sprintf('%s_FFT频谱.png', dev)));
112 close(gcf);
113 % ACF 自相关图
114 figure('Visible','off');
115 autocorr(daily_per_full, 'NumLags', min(400,n_y-1));
116 title(sprintf('设备%s 日均透水性自相关函数', dev));
117 saveas(gcf, fullfile('Figures', sprintf('%s_ACF.png', dev)));
118 close(gcf);
119 % 全局劣化速率
120 t = (1:length(daily_per_full));
121 b_glob = [ones(size(t)), t] \ daily_per_full;
122 glob_slope_year = b_glob(2) * 365;
123 % 分段线性模型构建
124 segment_starts = [t_start; m_dates + days(1)];
125 segment_ends = [m_dates - days(1); t_end];
126 num_seg = length(segment_starts);
127 slopes = []; p_start_fits = []; mid_months = [];
128 for k = 1:num_seg
129     mask = daily_dates >= segment_starts(k) & daily_dates <= segment_ends(k);
130     t_seg = daily_dates(mask); p_seg = daily_per_full(mask);
131     if length(t_seg) < 3, continue; end
132     days_rel = days(t_seg - segment_starts(k));
133     X_seg = [ones(size(days_rel)), days_rel];
134     b_seg = X_seg \ p_seg;
135     R2 = 1 - sum((p_seg - X_seg*b_seg).^2) / sum((p_seg - mean(p_seg)).^2);
136     slopes(end+1) = b_seg(2);
137     p_start_fits(end+1) = b_seg(1);
138     mid_date = segment_starts(k) + days(round(max(days_rel)/2));
139     mid_months(end+1) = month(mid_date);
140     % 保存分段信息
141     t_row = table({dev}, k, segment_starts(k), segment_ends(k), b_seg(2), R2, b_seg(1),
        month(mid_date), ...
142         'VariableNames', {'Device','Segment','StartDate','EndDate','Slope','R2','P_start',
        'MidMonth'});
143     all_seg_info = [all_seg_info; t_row];
144 end
145 if length(slopes) >= 2
146     figure('Visible','off');
147     % 左图: 斜率随时间散点图 + 线性拟合趋势
148     subplot(1,2,1);
149     mid_dates_seg = segment_starts(1:length(slopes)) + ...
150         days(round((segment_ends(1:length(slopes)) - segment_starts(1:length(
        slopes))))/2));
151     scatter(mid_dates_seg, slopes, 40, 'filled', 'MarkerFaceColor', [0.2 0.6 0.9]);
152     hold on;
153     % 线性拟合线
154     if length(slopes) >= 3
155         p_poly = polyfit(datenum(mid_dates_seg), slopes, 1);
156         t_fine = linspace(min(datenum(mid_dates_seg)), max(datenum(mid_dates_seg)), 100);
157         plot(datetime(t_fine,'ConvertFrom','datenum'), polyval(p_poly, t_fine), ...
158             'r-', 'color','#f36f43', 'LineWidth', 1.5);
159         title(sprintf('设备%s 分段斜率 (年加速度=%.4f)', dev, p_poly(1)*365));
160     else
161         title(sprintf('设备%s 分段斜率', dev));
162     end
163     xlabel('时间'); ylabel('日均劣化斜率 (单位/天)');
164     grid on; datetick('x','yyyy-mm','keeplimits');
165
166     % 右图: 按季节分色的散点图, 观察季节内变化
167     subplot(1,2,2);
168     if ~isempty(slopes)
169         % 定义季节颜色

```

```

170     season_color = zeros(length(slopes),3);
171     for si = 1:length(slopes)
172         m = mid_months(si);
173         if ismember(m,[3 4 5]), season_color(si,:) = [0.2 0.8 0.2]; % 春绿
174         elseif ismember(m,[6 7 8]), season_color(si,:) = [1.0 0.5 0.0]; % 夏橙
175         elseif ismember(m,[9 10 11]), season_color(si,:) = [0.8 0.2 0.2]; % 秋红
176         else, season_color(si,:) = [0.3 0.3 0.8]; % 冬蓝
177     end
178     end
179     % 横轴用段序号或日期
180     x_plot = 1:length(slopes);
181     scatter(x_plot, slopes, 40, season_color, 'filled');
182     hold on;
183     % 连接线以便观察走势
184     plot(x_plot, slopes, '-k', 'LineWidth', 0.5);
185     xlabel('分段序号'); ylabel('日均劣化斜率');
186     title('逐段斜率变化 (颜色:季节)');
187     legend({'春','夏','秋','冬'}, 'Location','best');
188     grid on;
189     end
190     saveas(gcf, fullfile('Figures', sprintf('%s_分段斜率演化.png', dev)));
191     close(gcf);
192 end
193 % 分段拟合图
194 figure('Visible','off');
195 plot(daily_dates, daily_per_full, 'Color', [0.7 0.7 0.7]); hold on;
196 for k = 1:num_seg
197     if k <= length(slopes) && ~isnan(slopes(k))
198         t_line = segment_starts(k):days(1):segment_ends(k);
199         days_line = days(t_line - segment_starts(k));
200         p_line = p_start_fits(k) + slopes(k) * days_line;
201         plot(t_line, p_line, 'r', 'LineWidth', 1.5);
202     end
203 end
204 xline(m_dates, '--k'); legend('日均透水率','分段拟合','维护日期'); grid on;
205 title(sprintf('设备%s 分段线性劣化模型', dev));
206 saveas(gcf, fullfile('Figures', sprintf('%s_分段拟合.png', dev)));
207 close(gcf);
208
209 % 基于斜率构建季节影响算子
210 if isempty(slopes)
211     base_slope = NaN; sp=NaN; su=NaN; au=NaN; wi=NaN;
212     sp_r=NaN; su_r=NaN; au_r=NaN; wi_r=NaN;
213 else
214     base_slope = mean(slopes);
215     season = cell(length(slopes),1);
216     for i = 1:length(slopes)
217         m = mid_months(i);
218         if ismember(m,[3 4 5]), season{i}='春';
219         elseif ismember(m,[6 7 8]), season{i}='夏';
220         elseif ismember(m,[9 10 11]), season{i}='秋';
221         else, season{i}='冬';
222     end
223     end
224     [G_season, s_names] = findgroups(season);
225     s_avg = splitapply(@mean, slopes', G_season);
226     sp=NaN; su=NaN; au=NaN; wi=NaN;
227     for si = 1:length(s_names)
228         switch s_names{si}
229             case '春', sp = s_avg(si);
230             case '夏', su = s_avg(si);
231             case '秋', au = s_avg(si);
232             case '冬', wi = s_avg(si);
233         end
234     end
235     sp_r = sp / base_slope; su_r = su / base_slope;
236     au_r = au / base_slope; wi_r = wi / base_slope;
237 end
238 % 劣化加速度构建分析下降规律
239 if length(slopes) >= 3
240     mid_dates_seg = segment_starts(1:length(slopes)) + days(round((segment_ends(1:length(
241     slopes))-segment_starts(1:length(slopes)))/2));
242     p_poly = polyfit(denum(mid_dates_seg), slopes, 1);
243     accel_year = p_poly(1) * 365;
244 else
245     accel_year = NaN;
246 end
247 % 季节斜率图
248 if ~isempty(slopes)
249     figure('Visible','off');
250     subplot(1,2,1);
251     boxchart(categorical(season), slopes);
252     ylabel('日均劣化速率'); title('各季节劣化速率分布'); grid on;

```



```

252     subplot(1,2,2);
253     scatter(mid_dates_seg, slopes, 30, 'filled'); hold on;
254     if ~isnan(accel_year)
255         t_fine = linspace(min(datenum(mid_dates_seg)), max(datenum(mid_dates_seg)), 100);
256         plot(datetime(t_fine, 'ConvertFrom', 'datenum'), polyval(p_poly, t_fine), 'r-', '
LineWidth', 1.5);
257     end
258     xlabel('时间'); ylabel('日均劣化速率');
259     title(sprintf('设备%s 劣化速率变化 (年变=%4f)', dev, accel_year)); grid on;
260     saveas(gcf, fullfile('Figures', sprintf('%s_季节斜率.png', dev)));
261     close(gcf);
262 end
263 % 维护顺势增益分析
264 before_days = 5; after_days = 60;
265 delta_vals = []; dur_vals = [];
266 for mi = 1:length(m_dates)
267     [~, id_main] = min(abs(daily_dates - m_dates(mi)));
268     pre_s = max(1, id_main - before_days); pre_e = id_main - 1;
269     post_s = id_main + 1; post_e = min(length(daily_per_full), id_main + after_days);
270     if pre_e < pre_s || post_e <= post_s, continue; end
271     P_pre = median(daily_per_full(pre_s:pre_e));
272     P_post = daily_per_full(post_s:post_e);
273     [P_peak, ~] = max(P_post);
274     delta_vals(end+1) = P_peak - P_pre;
275 end
276 mid_mask = (m_types(1:length(delta_vals)) == 1);
277 big_mask = (m_types(1:length(delta_vals)) == 2);
278 avg_delta_mid = mean(delta_vals(mid_mask), 'omitnan');
279 avg_delta_big = mean(delta_vals(big_mask), 'omitnan');
280
281 % 维护瞬时增益图
282 figure('Visible','off');
283 m_labels = {'中维护', '大维护'};
284 for t = 1:2
285     mask = (m_types(1:length(delta_vals)) == t);
286     delta = delta_vals(mask);
287     subplot(1,2,t);
288     if sum(mask) >= 2
289         boxplot(delta, 'Labels', {m_labels{t}});
290     elseif sum(mask) == 1
291         bar(delta); set(gca, 'XTickLabel', {m_labels{t}});
292         text(1, delta, num2str(delta), 'HorizontalAlignment','center','VerticalAlignment','
bottom');
293     else
294         text(0.5, 0.5, '无数据', 'HorizontalAlignment','center');
295     end
296     ylabel('ΔP'); title(m_labels{t});
297 end
298 saveas(gcf, fullfile('Figures', sprintf('%s_维护增益.png', dev)));
299 close(gcf);
300
301 % 维护损伤构建
302 first_mid_start = NaN; deg_mid = []; deg_big = [];
303 for k = 2:num_seg-1
304     possible_md = segment_starts(k) - days(1);
305     [tf, loc] = ismember(possible_md, m_dates);
306     if ~tf || k > length(p_start_fits), continue; end
307     m_type = m_types(loc);
308     cur_start = p_start_fits(k);
309     if k==2 && m_type==1, first_mid_start = cur_start; end
310     if ~isnan(first_mid_start) && ~isnan(cur_start)
311         deg = first_mid_start - cur_start;
312         if m_type==1, deg_mid(end+1)=deg; else, deg_big(end+1)=deg; end
313     end
314 end
315 avg_deg_mid = mean(deg_mid, 'omitnan');
316 avg_deg_big = mean(deg_big, 'omitnan');
317
318 % 数据汇总
319 summ.Device = dev;
320 summ.GlobalSlopeYear = glob_slope_year;
321 summ.SeasAmplitude = A_s;
322 summ.BaseSlope = base_slope;
323 summ.SpringSlope = sp; summ.SummerSlope = su;
324 summ.AutumnSlope = au; summ.WinterSlope = wi;
325 summ.SpringRatio = sp_r; summ.SummerRatio = su_r;
326 summ.AutumnRatio = au_r; summ.WinterRatio = wi_r;
327 summ.DeltaP_Mid = avg_delta_mid; summ.DeltaP_Big = avg_delta_big;
328 summ.DamageMid = avg_deg_mid; summ.DamageBig = avg_deg_big;
329 summ.AccelYear = accel_year;
330 all_summaries{end+1} = summ;
331
332 % 命令行输出

```

```

333     fprintf('    全局年均劣化速率: %.2f 单位/年\n', glob_slope_year);
334     fprintf('    季节振幅(水平): %.2f\n', A_s);
335     fprintf('    基准劣化斜率: %.4f 单位/天\n', base_slope);
336     fprintf('    季节影响算子: 春 %.2f, 夏 %.2f, 秋 %.2f, 冬 %.2f\n', sp_r, su_r, au_r, wi_r);
337     fprintf('    维护增益: 中 %.2f, 大 %.2f\n', avg_delta_mid, avg_delta_big);
338     fprintf('    损伤系数: 中 %.2f, 大 %.2f\n', avg_deg_mid, avg_deg_big);
339     fprintf('    劣化加速度: %.4f /年\n\n', accel_year);
340 end
341
342 % 绘制A1-A5的季节项分析图
343 if all(~cellfun(@isempty, season_cell))
344     figure('Visible','off');
345     t = tiledlayout(5,1, 'TileSpacing', 'compact', 'Padding', 'compact');
346     for i = 1:5
347         nexttile;
348         data_mat = season_cell{i};
349         plot(datetime(data_mat(:,1),'ConvertFrom','datenum'), data_mat(:,2), 'r', 'LineWidth'
350             ,0.8);
351         title(sprintf('设备 %s 季节项 (振幅=%.2f)', season_dev_names(i), season_amp(i)));
352         ylabel('透水率');
353         if i == 5
354             xlabel('日期');
355         end
356         grid on;
357     end
358     sgtitle('设备 A1~A5 季节分量对比');
359     saveas(gcf, fullfile('Figures', 'A1_A5_Seasonal_Comparison.png'));
360     close(gcf);
361 end
362 % 设备分段斜率汇总图
363 if ~isempty(all_seg_info)
364     figure('Visible','off');
365     devices = unique(string(all_seg_info.Device));
366     cmap = lines(length(devices));
367     hold on;
368     for di = 1:length(devices)
369         dev_name = devices(di);
370         idx_dev = strcmp(all_seg_info.Device, dev_name);
371         dev_slopes = all_seg_info.Slope(idx_dev);
372         dev_dates = all_seg_info.StartDate(idx_dev) + ...
373             (all_seg_info.EndDate(idx_dev) - all_seg_info.StartDate(idx_dev))/2;
374         h = scatter(dev_dates, dev_slopes, 30, cmap(di,:), 'filled', ...
375             'DisplayName', char(dev_name));
376         plot(dev_dates, dev_slopes, '-', 'Color', cmap(di,:), 'LineWidth', 0.5, ...
377             'HandleVisibility','off');
378     end
379     xlabel('时间'); ylabel('日均劣化斜率 (单位/天)');
380     title('10台设备分段劣化斜率演变');
381     legend('show','Location','best');
382     grid on;
383     datetick('x','yyyy-mm','keeplimits');
384     saveas(gcf, fullfile('Figures', 'All_Devices_Slopes_Overview.png'));
385     close(gcf);
386 end
387 %数据表格汇总
388 if isempty(all_summaries)
389     error('无有效设备分析结果!');
390 end
391 T_all = struct2table([all_summaries{:}]);
392 T_all = sortrows(T_all, 'Device');
393 disp('10台设备综合指标汇总');
394 disp(T_all);
395 writetable(T_all, 'All_Devices_Summary.xlsx');
396 if ~isempty(all_seg_info)
397     writetable(all_seg_info, 'All_Segment_Params.xlsx');
398 end
399 % 四季平均斜率汇总表
400 season_table = T_all(:, {'Device', 'SpringSlope', 'SummerSlope', 'AutumnSlope', 'WinterSlope'});
401 avg_spring = mean(T_all.SpringSlope, 'omitnan');
402 avg_summer = mean(T_all.SummerSlope, 'omitnan');
403 avg_autumn = mean(T_all.AutumnSlope, 'omitnan');
404 avg_winter = mean(T_all.WinterSlope, 'omitnan');
405 avg_row = table({'平均'}, avg_spring, avg_summer, avg_autumn, avg_winter, ...
406     'VariableNames', season_table.Properties.VariableNames);
407 season_table = [season_table; avg_row];
408 writetable(season_table, 'Season_Slopes.xlsx');
409 disp('四季平均斜率已保存至 Season_Slopes.xlsx');
410
411 % 各设备年度劣化斜率汇总表
412 if ~isempty(all_seg_info)
413     mid_dates_all = all_seg_info.StartDate + ...
414         (all_seg_info.EndDate - all_seg_info.StartDate)/2;
415     m = month(mid_dates_all);

```

```

415     y = year(mid_dates_all);
416
417     fiscal_start = y;
418     fiscal_start(m < 4) = y(m < 4) - 1;
419     fiscal_start(fiscal_start >= 2026) = 2025;
420
421     % 生成带月份的区间标签
422     year_label = strings(size(fiscal_start));
423     for i = 1:length(fiscal_start)
424         if fiscal_start(i) == 2025
425             year_label(i) = "2025.4-2026.4";
426         else
427             year_label(i) = sprintf("%d.4-%d.3", fiscal_start(i), fiscal_start(i)+1);
428         end
429     end
430
431     % 分组求平均斜率
432     dev_str = string(all_seg_info.Device);
433     [G, dev_grp, yr_grp] = findgroups(dev_str, year_label);
434     avg_slope_yearly = splitapply(@mean, all_seg_info.Slope, G);
435
436     devices_unique = unique(dev_str, 'stable');
437     all_labels = unique(year_label);
438     % 按起始年份排序
439     start_years = str2double(extractBefore(all_labels, '.'));
440     [~, idx] = sort(start_years);
441     years_unique = all_labels(idx);
442
443     yearly_matrix = NaN(length(years_unique), length(devices_unique));
444     for i = 1:length(dev_grp)
445         d_idx = find(strcmp(devices_unique, dev_grp{i}));
446         y_idx = find(strcmp(years_unique, yr_grp{i}));
447         yearly_matrix(y_idx, d_idx) = avg_slope_yearly(i);
448     end
449
450     YearlyTable = array2table(yearly_matrix, ...
451         'VariableNames', matlab.lang.makeValidName(cellstr(devices_unique)));
452     YearlyTable = addvars(YearlyTable, years_unique, 'Before', 1, ...
453         'NewVariableNames', {'Year'});
454
455     writetable(YearlyTable, 'Yearly_Slopes.xlsx');
456     disp('年度劣化斜率（含具体月份区间）已保存至 Yearly_Slopes.xlsx');
457 end
458
459 %辅助函数：经典季节分解
460 function [trend, seasonal, residual] = classical_seasonal_decomp(x, period)
461     n = length(x);
462     trend = movmean(x, [period, 0]);
463     detrended = x - trend;
464     seasonal_pattern = zeros(period,1);
465     for i = 1:period
466         idx = i:period:n;
467         seasonal_pattern(i) = mean(detrended(idx));
468     end
469     seasonal_pattern = seasonal_pattern - mean(seasonal_pattern);
470     seasonal = repmat(seasonal_pattern, ceil(n/period), 1);
471     seasonal = seasonal(1:n);
472     residual = x - trend - seasonal;
473 end

```

code/Q1_STL.m

Q1_Parameter.m

```

1  clear; clc; close all;
2  %数据导入
3  maint_file = '附件2整合数据.xlsx';
4  maint = detectImportOptions(maint_file);
5  maint = setvartype(maint, {'ID','DATE'}, 'string');
6  maint_data = readtable(maint_file, maint);
7  maint_data.DATE = datetime(maint_data.DATE, 'InputFormat', 'yyyy-MM-dd');
8  maint_data.TypeNum = zeros(height(maint_data),1);
9  maint_data.TypeNum(contains(maint_data.LEIBIE,'中')) = 1;
10 maint_data.TypeNum(contains(maint_data.LEIBIE,'大')) = 2;
11
12 device_ids = unique(maint_data.ID);
13 all_device_params = struct();
14 all_maint_events = {};
15
16 %设备参数提取
17 for d = 1:length(device_ids)
18     dev = device_ids(d);

```

```

19 % 数据导入
20 dev_mod = regexprep(dev, '([A-Za-z])(\d)', '$1_$2');
21 data_file = sprintf('设备%s_清洗结果_含缺失填补.xlsx', dev_mod);
22 if ~isfile(data_file), continue; end
23 data = readtable(data_file);
24 data.Time = datetime(data.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
25 [dates_u, ~, idx_u] = unique(dateshift(data.Time, 'start', 'day'));
26 daily_per = accumarray(idx_u, data.final_per, [], @mean);
27 daily_per = fillmissing(daily_per, 'linear');
28 % 斜率提取
29 idx_mt = maint_data.ID == dev;
30 m_dates = maint_data.DATE(idx_mt);
31 m_types = maint_data.TypeNum(idx_mt);
32 [m_dates, s_idx] = sort(m_dates); m_types = m_types(s_idx);
33 seg_starts = [dates_u(1); m_dates + days(1)];
34 seg_ends = [m_dates - days(1); dates_u(end)];
35 slopes = []; mid_months = []; gain_mid = []; gain_big = [];
36 for k = 1:length(seg_starts)
37     mask = dates_u >= seg_starts(k) & dates_u <= seg_ends(k);
38     if sum(mask) < 5, continue; end
39     t_vec = (0:sum(mask)-1)';
40     b = [ones(size(t_vec)), t_vec] \ daily_per(mask);
41     slopes(end+1) = b(2);
42     mid_date = seg_starts(k) + days(round(sum(mask)/2));
43     mid_months(end+1) = month(mid_date);
44     % 辨识增益 Delta P
45     if k > 1
46         [~, tidx] = min(abs(dates_u - m_dates(k-1)));
47         dp = mean(daily_per(tidx:min(tidx+3, end))) - mean(daily_per(max(1, tidx-3):tidx));
48         if m_types(k-1) == 1
49             gain_mid(end+1) = dp;
50         else
51             gain_big(end+1) = dp;
52         end
53     % 维护事件分析
54     maint_type = m_types(k-1);
55     maint_month = month(m_dates(k-1));
56
57     all_maint_events(end+1, :) = {char(dev), maint_type, dp, maint_month};
58 end
59 end
60 % 季节算子
61 k_avg = mean(slopes);
62 S_vals = ones(1,4);
63 season_idx = zeros(size(mid_months));
64 for i = 1:length(mid_months)
65     m = mid_months(i);
66     if ismember(m,[3 4 5]), season_idx(i)=1;
67     elseif ismember(m,[6 7 8]), season_idx(i)=2;
68     elseif ismember(m,[9 10 11]), season_idx(i)=3;
69     else, season_idx(i)=4; end
70 end
71 for s = 1:4
72     if any(season_idx == s), S_vals(s) = mean(slopes(season_idx==s)) / k_avg; end
73 end
74
75 % 链式损伤α
76 a_mid = []; a_big = [];
77 for i = 2:length(slopes)
78     kp_prev = abs(slopes(i-1) / S_vals(season_idx(i-1)));
79     kp_curr = abs(slopes(i) / S_vals(season_idx(i)));
80     if kp_prev > 1e-6
81         alpha = (kp_curr - kp_prev) / kp_prev;
82         if m_types(i-1) == 1, a_mid(end+1) = alpha; else, a_big(end+1) = alpha; end
83     end
84 end
85
86 % 结果输出
87 fprintf('结果输出: %s\n', dev);
88 fprintf('    基础劣化率 k0:    %.5f \n', abs(k_avg));
89 fprintf('    中维护损伤 Alpha_M:    %.4f \n', max(0, mean(a_mid, 'omitnan')));
90 fprintf('    大维护损伤 Alpha_B:    %.4f \n', max(0, mean(a_big, 'omitnan')));
91 fprintf('    中维护瞬时增益:    +%.2f\n', mean(gain_mid, 'omitnan'));
92 fprintf('    大维护瞬时增益:    +%.2f\n', mean(gain_big, 'omitnan'));
93 fprintf('    季节算子: 春%.2f 夏%.2f 秋%.2f 冬%.2f\n', S_vals);
94 res.k0 = abs(k_avg); res.S = S_vals;
95 res.am = mean(a_mid); res.ab = mean(a_big);
96 res.gm = mean(gain_mid); res.gb = mean(gain_big);
97 all_device_params.(char(dev)) = res;
98 end
99 % 数据表格化
100 devNames = fieldnames(all_device_params);
101 nDev = length(devNames);

```

```

102 gm_all = NaN(nDev,1); gb_all = NaN(nDev,1);
103 am_all = NaN(nDev,1); ab_all = NaN(nDev,1);
104
105 for i = 1:nDev
106     dev = devNames{i};
107     gm_all(i) = all_device_params.(dev).gm;
108     gb_all(i) = all_device_params.(dev).gb;
109     am_all(i) = max(0, all_device_params.(dev).am);
110     ab_all(i) = max(0, all_device_params.(dev).ab);
111 end
112 diff_gain = gm_all - gb_all;
113 diff_alpha = am_all - ab_all;
114
115 gainTable = table(devNames, gm_all, gb_all, diff_gain, ...
116     'VariableNames', {'设备ID', '中维护增益', '大维护增益', '中维护减小维护'});
117 avg_gm = mean(gm_all, 'omitnan'); avg_gb = mean(gb_all, 'omitnan');
118 gainTable_avg = table({'平均'}, avg_gm, avg_gb, avg_gm - avg_gb, ...
119     'VariableNames', {'设备ID', '中维护增益', '大维护增益', '中维护减小维护'});
120 gainTable = [gainTable; gainTable_avg];
121
122 alphaTable = table(devNames, am_all, ab_all, diff_alpha, ...
123     'VariableNames', {'设备ID', '中维护损伤', '大维护损伤', '中维护减小维护'});
124 avg_am = mean(am_all, 'omitnan'); avg_ab = mean(ab_all, 'omitnan');
125 alphaTable_avg = table({'平均'}, avg_am, avg_ab, avg_am - avg_ab, ...
126     'VariableNames', {'设备ID', '中维护损伤', '大维护损伤', '中维护减小维护'});
127 alphaTable = [alphaTable; alphaTable_avg];
128
129 fprintf('\n%s\n', repmat('-',1,80));
130 disp('维护增益数据');
131 disp(gainTable);
132 fprintf('\n%s\n', repmat('-',1,80));
133 disp('维护损伤数据');
134 disp(alphaTable);
135
136 %数据转化为表格输出
137 if isempty(all_maint_events)
138     error('未收集到任何维护事件，请检查数据!');
139 end
140 event_table = cell2table(all_maint_events, ...
141     'VariableNames', {'DeviceID', 'TypeNum', 'Gain', 'Month'});
142 event_table.DeviceID = string(event_table.DeviceID);
143 % 添加季节标签
144 season_label = strings(height(event_table), 1);
145 for i = 1:height(event_table)
146     m = event_table.Month(i);
147     if ismember(m, [3 4 5])
148         season_label(i) = "春";
149     elseif ismember(m, [6 7 8])
150         season_label(i) = "夏";
151     elseif ismember(m, [9 10 11])
152         season_label(i) = "秋";
153     else
154         season_label(i) = "冬";
155     end
156 end
157 event_table.Season = season_label;
158 devices = unique(event_table.DeviceID, 'stable');
159 seasons = ["春", "夏", "秋", "冬"];
160 nDev = length(devices);
161 nSea = length(seasons);
162 % Sheet1: 全部维护
163 gain_all = NaN(nDev, nSea);
164 for i = 1:nDev
165     for j = 1:nSea
166         idx = event_table.DeviceID == devices(i) & event_table.Season == seasons(j);
167         if any(idx), gain_all(i,j) = mean(event_table.Gain(idx), 'omitnan'); end
168     end
169 end
170 avg_all = mean(gain_all, 1, 'omitnan');
171 T_all = array2table([gain_all; avg_all], ...
172     'RowNames', [cellstr(devices); '平均'], 'VariableNames', cellstr(seasons));
173 % Sheet2: 中维护
174 gain_mid = NaN(nDev, nSea);
175 for i = 1:nDev
176     for j = 1:nSea
177         idx = event_table.DeviceID == devices(i) & event_table.Season == seasons(j) &
178             event_table.TypeNum == 1;
179         if any(idx), gain_mid(i,j) = mean(event_table.Gain(idx), 'omitnan'); end
180     end
181 end
182 avg_mid = mean(gain_mid, 1, 'omitnan');
183 T_mid = array2table([gain_mid; avg_mid], ...
184     'RowNames', [cellstr(devices); '平均'], 'VariableNames', cellstr(seasons));

```

```

184 % Sheet3: 大维护
185 gain_big = NaN(nDev, nSea);
186 for i = 1:nDev
187     for j = 1:nSea
188         idx = event_table.DeviceID == devices(i) & event_table.Season == seasons(j) &
            event_table.TypeNum == 2;
189         if any(idx), gain_big(i,j) = mean(event_table.Gain(idx), 'omitnan'); end
190     end
191 end
192 avg_big = mean(gain_big, 1, 'omitnan');
193 T_big = array2table([gain_big; avg_big], ...
194     'RowNames', [cellstr(devices); '平均'], 'VariableNames', cellstr(seasons));
195 % 转成Excel
196 writetable(T_all, '季节维护增益.xlsx', 'Sheet', '全部维护', 'WriteRowNames', true);
197 writetable(T_mid, '季节维护增益.xlsx', 'Sheet', '中维护', 'WriteRowNames', true);
198 writetable(T_big, '季节维护增益.xlsx', 'Sheet', '大维护', 'WriteRowNames', true);

```

code/Q1_Parameter.m

Q2_Finalist.m

```

1 clear; clc; close all;
2 %数据环境导入
3 maint_file = '附件2整合数据.xlsx';
4 maint = detectImportOptions(maint_file);
5 maint = setvartype(maint, {'ID','DATE'}, 'string');
6 maint_data = readtable(maint_file, maint);
7 maint_data.DATE = datetime(maint_data.DATE, 'InputFormat', 'yyyy-MM-dd');
8 maint_data.TypeNum = zeros(height(maint_data),1);
9 maint_data.TypeNum(contains(maint_data.LEIBIE,'中')) = 1;
10 maint_data.TypeNum(contains(maint_data.LEIBIE,'大')) = 2;
11 t_born = datetime(2022,4,1);
12 device_ids = unique(maint_data.ID);
13
14 %设备辨识绘图
15 for d = 1:length(device_ids)
16     dev = device_ids(d);
17     % 实测数据载入
18     dev_mod = regexprep(dev, '([A-Za-z])(\d)', '$1_$2');
19     data_file = sprintf('设备%s_清洗结果_含缺失填补.xlsx', dev_mod);
20     if ~isfile(data_file), continue; end
21     data = readtable(data_file);
22     data.Time = datetime(data.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
23     [dates_u,~,idx_u] = unique(dateshift(data.Time,'start','day'));
24     daily_per = accumarray(idx_u, data.final_per,[], @mean);
25     daily_per = fillmissing(daily_per,'linear');
26     %维护数据提取
27     idx_mt = (maint_data.ID == dev);
28     m_dates = maint_data.DATE(idx_mt);
29     m_types = maint_data.TypeNum(idx_mt);
30     [m_dates, s_idx_sort] = sort(m_dates);
31     m_types = m_types(s_idx_sort);
32     %参数识别确认
33     seg_starts = [dates_u(1); m_dates + days(1)];
34     seg_ends = [m_dates - days(1); dates_u(end)];
35     slopes = []; mid_months = []; gain_mid = []; gain_big = [];
36     for k = 1:length(seg_starts)
37         mask = dates_u >= seg_starts(k) & dates_u <= seg_ends(k);
38         if sum(mask) < 5, continue; end
39         % 分段线性拟合斜率
40         t_vec = (0:sum(mask)-1)';
41         b = [ones(size(t_vec)), t_vec] \ daily_per(mask);
42         slopes(end+1) = b(2); % 斜率, 负值表示下降
43         % 段中点月份
44         mid_date = seg_starts(k) + days(round(sum(mask)/2));
45         mid_months(end+1) = month(mid_date);
46         % 维护瞬时增益
47         if k > 1
48             [~, tidx] = min(abs(dates_u - m_dates(k-1)));
49             dp = mean(daily_per(tidx:min(tidx+3, end))) - mean(daily_per(max(1, tidx-3):tidx));
50             if m_types(k-1) == 1
51                 gain_mid(end+1) = dp;
52             else
53                 gain_big(end+1) = dp;
54             end
55         end
56     end
57     % 基础劣化率 k0
58     k_avg = mean(slopes);
59     res_k0 = abs(k_avg); % 取正值为劣化速率
60     % 季节算子 S (春季=3-5, 夏季=6-8, 秋季=9-11, 冬季=12-2)

```

```

61 S_vals = ones(1,4);
62 season_idx = zeros(size(mid_months));
63 for i = 1:length(mid_months)
64     mv = mid_months(i);
65     if ismember(mv,[3 4 5])
66         season_idx(i) = 1;
67     elseif ismember(mv,[6 7 8])
68         season_idx(i) = 2;
69     elseif ismember(mv,[9 10 11])
70         season_idx(i) = 3;
71     else
72         season_idx(i) = 4;
73     end
74 end
75 for s = 1:4
76     if any(season_idx == s)
77         S_vals(s) = mean(slopes(season_idx == s)) / k_avg;
78     end
79 end
80 % 链式损伤 Alpha
81 a_mid = []; a_big = [];
82 for i = 2:length(slopes)
83     kp_prev = abs(slopes(i-1) / S_vals(season_idx(i-1)));
84     kp_curr = abs(slopes(i) / S_vals(season_idx(i)));
85     if kp_prev > 1e-6
86         alpha = (kp_curr - kp_prev) / kp_prev;
87         if m_types(i-1) == 1
88             a_mid(end+1) = alpha;
89         else
90             a_big(end+1) = alpha;
91         end
92     end
93 end
94 % 维护增益
95 res_gm = mean(gain_mid, 'omitnan');
96 res_gb = mean(gain_big, 'omitnan');
97 if isnan(res_gm), res_gm = 5; end % 缺省值保护
98 if isnan(res_gb), res_gb = 12; end
99 res_gm = max(5, res_gm); % 保留原有仿真约束
100 res_gb = max(12, res_gb);
101 % 损伤系数均值
102 alpha_mid_mean = mean(a_mid, 'omitnan');
103 alpha_big_mean = mean(a_big, 'omitnan');
104 if isnan(alpha_mid_mean), alpha_mid_mean = 0; end
105 if isnan(alpha_big_mean), alpha_big_mean = 0; end
106 % 历史维护次数统计 (新增)
107 hist_mid = sum(m_types == 1);
108 hist_big = sum(m_types == 2);
109 %结果输出
110 fprintf('结果输出: %s\n', dev);
111 fprintf(' 1、劣化与损伤系数:\n');
112 fprintf(' (1) 基础劣化率 k0: %.5f \n', res_k0);
113 fprintf(' (2) 中维护损伤 Alpha_M: %.4f \n', alpha_mid_mean);
114 fprintf(' (3) 大维护损伤 Alpha_B: %.4f \n', alpha_big_mean);
115 fprintf('\n 2、维护维护复增益 (Delta P):\n');
116 fprintf(' (1) 中维护瞬时增益: +%.2f %\n', res_gm);
117 fprintf(' (2) 大维护瞬时增益: +%.2f %\n', res_gb);
118 fprintf('\n 3、季节算子 (S, 基准=1.0):\n');
119 fprintf(' 春季: %.2f | 夏季: %.2f | 秋季: %.2f | 冬季: %.2f \n', ...
120     S_vals(1), S_vals(2), S_vals(3), S_vals(4));
121 fprintf('\n >> 历史维护次数:\n');
122 fprintf(' 中维护(历史): %d 次\n', hist_mid);
123 fprintf(' 大维护(历史): %d 次\n', hist_big);
124 %回溯计算分析
125 back_len = days(dates_u(1) - t_born);
126 if back_len > 0
127     p_back = zeros(back_len, 1);
128     temp_P = daily_per(1);
129     for i = 1:back_len
130         dt = dates_u(1) - days(i);
131         mv = month(dt);
132         s_idx = get_s_idx(mv);
133         k_t = res_k0 * S_vals(s_idx);
134         gain_val = 0;
135         m_idx = find(m_dates == dt, 1);
136         if ~isempty(m_idx)
137             if m_types(m_idx) == 1
138                 gain_val = res_gm;
139             else
140                 gain_val = res_gb;
141             end
142         end
143     end
144     temp_P = temp_P + k_t - gain_val; % 回溯: 加衰退量, 减增益

```



```

144         if temp_P > 140
145             temp_P = 140 + rand(); % 轻微随机限制
146         end
147         p_back(i) = temp_P;
148     end
149     value_series = [flipud(p_back); daily_per];
150 else
151     value_series = daily_per;
152 end
153 %寿命预测
154 curr_P = daily_per(end);
155 curr_date = dates_u(end);
156 time_series = (t_born : days(1) : curr_date)';
157 h_win = value_series(max(1, end-364):end);
158 prev_avg = mean(h_win);
159 has_big_config = any(m_types == 2);
160 emergency_used = false;
161 below_counter = 0;
162 is_dead = false;
163 % 新增：模拟维护计数器
164 sim_mid = 0;
165 sim_big = 0;
166 while ~is_dead && curr_date < (t_born + years(25))
167     curr_date = curr_date + days(1);
168     mv = month(curr_date);
169     s_idx = get_s_idx(mv);
170     k_t = res_k0 * S_vals(s_idx);
171     curr_P = curr_P - k_t; % 每日自然衰退
172     % 定期中维护（每60天自动增益）
173     if mod(days(curr_date - dates_u(end)), 60) == 0
174         curr_P = curr_P + res_gm;
175         sim_mid = sim_mid + 1; % 记录一次模拟中维护
176     end
177     %滚动年均值计算
178     value_series(end+1) = curr_P;
179     time_series(end+1) = curr_date;
180     h_win(end+1) = curr_P;
181     if length(h_win) > 365, h_win(1) = []; end
182     curr_avg = mean(h_win);
183     % 滚动年均值跌落触发应急大维护
184     if (prev_avg >= 37) && (curr_avg < 37)
185         if has_big_config && ~emergency_used
186             curr_P = curr_P + res_gb;
187             emergency_used = true;
188             sim_big = sim_big + 1; % 记录一次模拟大维护
189             value_series(end) = curr_P;
190             h_win(end) = curr_P;
191             curr_avg = mean(h_win);
192         end
193         below_counter = 1;
194     elseif curr_avg < 37
195         below_counter = below_counter + 1;
196     else
197         below_counter = 0;
198     end
199
200     if below_counter >= 30 || curr_P < 0
201         is_dead = true;
202     end
203     prev_avg = curr_avg;
204 end
205 % 总维护次数
206 total_mid = hist_mid + sim_mid;
207 total_big = hist_big + sim_big;
208 %寿命仿真结果输出
209 fprintf('\n 4、寿命预测结果\n')
210 fprintf('(1)回溯出厂 P0: %.2f%\n', value_series(1));
211 fprintf('    预计报废日期: %s\n', datestr(curr_date, 'yyyy-mm-dd'));
212 fprintf('    总运行寿命: %.2f 年\n', days(curr_date - t_born)/365.25);
213 % 经济性分析
214 purchase_cost = 3000000; % 设备购买成本 (元)
215 mid_cost = 30000; % 中维护单次成本 (元/次)
216 big_cost = 120000; % 大维护单次成本 (元/次)
217 total_cost = purchase_cost + total_mid * mid_cost + total_big * big_cost;
218 life_years = days(curr_date - t_born) / 365.25;
219 avg_annual_cost = total_cost / life_years;
220 fprintf('\n 5、经济性分析:\n');
221 fprintf('(1) 过滤器购买成本: %.0f 元\n', purchase_cost);
222 fprintf('(2) 中维护总次数: %d 次, 成本: %.2f 万元\n', total_mid, total_mid * mid_cost / 10000);
223 fprintf('(3) 大维护总次数: %d 次, 成本: %.2f 万元\n', total_big, total_big * big_cost / 10000);
224 fprintf('(4) 总成本: %.2f 万元\n', total_cost / 10000);

```



```

225     fprintf('(5) 总运行寿命: %.2f 年\n', life_years);
226     fprintf('(6) 年平均成本: %.2f 元/年\n', avg_annual_cost);
227     fprintf('%s\n\n', repmat('=',1,70));
228 %生命周期图绘制
229 [~, start_plot_idx] = min(abs(time_series - dates_u(1)));
230 t_plot = time_series(start_plot_idx:end);
231 v_plot = value_series(start_plot_idx:end);
232 N_plot = min(length(t_plot), length(v_plot));
233 t_plot = t_plot(1:N_plot);
234 v_plot = v_plot(1:N_plot);
235 full_mov_avg = movmean(value_series, [364 0], 'omitnan');
236 ma_plot = full_mov_avg(start_plot_idx:end);
237 ma_plot = ma_plot(1:N_plot);
238 figure('Color','w','Name',char(dev)); hold on; grid on;
239 plot(t_plot, v_plot, 'r-', 'color','#a30543','LineWidth', 0.8, 'DisplayName', '瞬时透水率');
240 plot(t_plot, ma_plot, 'k-', 'LineWidth', 1.5, 'DisplayName', '365d均值判定线');
241 yline(37, 'b--', 'LineWidth', 1.2, 'DisplayName', '阈值 37%');
242 datetick('x','yy-mm'); ylabel('透水率 (%)');
243 title(['过滤器 ', char(dev), ' 生命周期图 ']);
244 legend('Location','best');
245 end
246 function s = get_s_idx(mv)
247     if ismember(mv,[3 4 5])
248         s = 1; % 春
249     elseif ismember(mv,[6 7 8])
250         s = 2; % 夏
251     elseif ismember(mv,[9 10 11])
252         s = 3; % 秋
253     else
254         s = 4; % 冬
255     end
256 end

```

code/Q2_Finalist.m

Q2_Reliability.m

```

1 clear; clc; close all;
2 %数据导入
3 maint_file = '附件2整合数据.xlsx';
4 lujing = detectImportOptions(maint_file);
5 lujing = setvartype(lujing, {'ID','DATE'}, 'string');
6 data = readtable(maint_file, lujing);
7 data.DATE = datetime(data.DATE, 'InputFormat', 'yyyy-MM-dd');
8 data.TypeNum = zeros(height(data),1);
9 data.TypeNum(contains(data.LEIBIE,'中')) = 1;
10 data.TypeNum(contains(data.LEIBIE,'大')) = 2;
11 t_born = datetime(2022,4,1);
12 device_ids = unique(data.ID);
13 results_table = table();
14 %参数确认
15 for d = 1:length(device_ids)
16     dev = device_ids(d);
17     dev_mod = regexprep(dev,'([A-Za-z])(\d)','$1_$2');
18     data_file = sprintf('设备%s清洗结果_含缺失填补.xlsx',dev_mod);
19     if ~isfile(data_file), continue; end
20     raw_data = readtable(data_file);
21     raw_data.Time = datetime(raw_data.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
22     [dates_u,~,idx_u] = unique(dateshift(raw_data.Time,'start','day'));
23     daily_per = accumarray(idx_u,raw_data.final_per,[],@mean);
24     daily_per = fillmissing(daily_per,'linear');
25
26 % 数据切分
27 % 训练集: 前70%用于辨识参数;
28 % 测试集: 后30%用于验证预测准确性
29 split_idx = round(length(daily_per) * 0.7);
30 train_per = daily_per(1:split_idx);
31 test_per = daily_per(split_idx+1:end);
32 test_dates = dates_u(split_idx+1:end);
33
34 %参数识别-
35 idx_mt = (data.ID == dev);
36 m_dates = data.DATE(idx_mt);
37 m_types = data.TypeNum(idx_mt);
38 % 识别劣化率k0、季节算子S、维护增益G
39 [k0_est, S_est, Gm_est, Gb_est] = identify_params(dates_u(1:split_idx), train_per, m_dates, m_types);
40 %可靠性指标——误差分析
41 pred_test = zeros(length(test_per), 1);
42 curr_v = train_per(end);
43 for i = 1:length(test_per)

```

```

44     dt = test_dates(i);
45     % 演化步进
46     k_t = k0_est * S_est(get_s_idx(month(dt)));
47     curr_v = curr_v - k_t;
48     % 补偿逻辑
49     midx = find(m_dates == dt, 1);
50     if isempty(midx)
51         if m_types(midx)==1, curr_v = curr_v + Gm_est; else, curr_v = curr_v + Gb_est; end
52     end
53     pred_test(i) = curr_v;
54 end
55
56 mape = mean(abs((pred_test - test_per)./test_per)) * 100;
57 rmse = sqrt(mean((pred_test - test_per).^2));
58 % 物理一致性回溯分析
59 p_back = daily_per(1);
60 back_days = days(dates_u(1) - t_born);
61 for i = 1:back_days
62     dt = dates_u(1) - days(i);
63     k_t = k0_est * S_est(get_s_idx(month(dt)));
64     g_val = 0;
65     midx = find(m_dates == dt, 1);
66     if isempty(midx)
67         if m_types(midx)==1, g_val = Gm_est; else, g_val = Gb_est; end
68     end
69     p_back = p_back + k_t - g_val;
70     if p_back > 101.5, p_back = 100 + rand(); end
71 end
72 p0_val = p_back;
73
74 % 结果展示
75 res_entry = table(dev, mape, rmse, p0_val, 'VariableNames',{'ID','MAPE_pct','RMSE','P0_Backtrace'});
76 results_table = [results_table; res_entry];
77
78 % 过滤器预测拟合结果分析
79 if d <= 10
80     figure('Color','w','Name',[ 'Reliability_', char(dev)]); hold on; grid on;
81     plot(test_dates, test_per, 'r.', 'Color','#4965b0', 'MarkerSize', 10, 'DisplayName', '实际观测点(测试集)');
82     plot(test_dates, pred_test, 'r-', 'Color','#a30543', 'LineWidth', 1.5, 'DisplayName', '模型预测走势');
83     legend('Location','best'); datetick('x','mm-dd');
84 end
85 end
86
87 %结果输出
88 fprintf('\n模型可靠性检验汇总 \n');
89 disp(results_table);
90 fprintf('平均预测误差 MAPE: %.2f%%\n', mean(results_table.MAPE_pct));
91 fprintf('平均回溯初始值 P0: %.2f%%\n', mean(results_table.P0_Backtrace))
92
93 %参数确认函数
94 function [k0, S, Gm, Gb] = identify_params(t_vec, v_vec, m_dates, m_types)
95     seg_starts = [t_vec(1); m_dates(m_dates>=t_vec(1) & m_dates<=t_vec(end))+days(1)];
96     seg_ends = [m_dates(m_dates>=t_vec(1) & m_dates<=t_vec(end))-days(1); t_vec(end)];
97     slopes = []; s_idx_list = []; g_m = []; g_b = [];
98     for k = 1:length(seg_starts)
99         mask = t_vec >= seg_starts(k) & t_vec <= seg_ends(k);
100         if sum(mask) < 3, continue; end
101         b = [ones(sum(mask),1), (0:sum(mask)-1)'] \ v_vec(mask);
102         slopes(end+1) = abs(b(2));
103         s_idx_list(end+1) = get_s_idx(month(seg_starts(k)));
104         if k > 1
105             [~, tidx] = min(abs(t_vec - m_dates(k-1)));
106             gain = mean(v_vec(tidx:min(tidx+2,end))) - mean(v_vec(max(1,tidx-2):tidx));
107             if m_types(k-1)==1, g_m(end+1)=gain; else, g_b(end+1)=gain; end
108         end
109     end
110     k0 = mean(slopes);
111     S = ones(1,4); for s=1:4, if any(s_idx_list==s), S(s)=mean(slopes(s_idx_list==s))/k0; end;
112     end
113     Gm = max(5, mean(g_m,'omitnan')); Gb = max(12, mean(g_b,'omitnan'));
114 end
115
116 function s = get_s_idx(mv)
117     if ismember(mv,[3 4 5]), s=1; elseif ismember(mv,[6 7 8]), s=2;
118     elseif ismember(mv,[9 10 11]), s=3; else, s=4; end
119 end

```

code/Q2_Reliability.m

Q3Q4.m

```

1  %%第三、四问：维护策略优化模型与参数敏感性分析
2  clear; clc; close all;
3  rng(0); % 固定随机种子
4  %读取维护数据
5  maint_file = '附件2整合数据.xlsx';
6  opts_m = detectImportOptions(maint_file);
7  opts_m = setvartype(opts_m, {'ID','DATE'}, 'string');
8  maint_data = readtable(maint_file, opts_m);
9  maint_data.DATE = datetime(maint_data.DATE, 'InputFormat', 'yyyy-MM-dd');
10 maint_data.TypeNum = zeros(height(maint_data),1);
11 maint_data.TypeNum(contains(maint_data.LEIBIE,'中')) = 1;
12 maint_data.TypeNum(contains(maint_data.LEIBIE,'大')) = 2;
13 t_born = datetime(2022,4,1);
14 device_ids = unique(maint_data.ID);
15 final_table = [];
16 dev_params = {};
17 %%维护策略优化循环
18 for d = 1:length(device_ids)
19     dev = device_ids(d);
20     fprintf('\n正在优化设备 %s...\n', char(dev));
21     dev_mod = regexprep(dev,'([A-Za-z])(\d)','$1_$2');
22     data_file = sprintf('设备%s_清洗结果_含缺失填补.xlsx', dev_mod);
23     if ~isfile(data_file), continue; end
24     data = readtable(data_file);
25     data.Time = datetime(data.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');
26     [dates_u,~,idx_u] = unique(dateshift(data.Time,'start','day'));
27     daily_per = accumarray(idx_u, data.final_per,[],@mean);
28     daily_per = fillmissing(daily_per,'linear');
29     idx_mt = (maint_data.ID == dev);
30     m_dates = maint_data.DATE(idx_mt);
31     m_types = maint_data.TypeNum(idx_mt);
32     [m_dates, s_idx_sort] = sort(m_dates);
33     m_types = m_types(s_idx_sort);
34     %%参数辨识（继承第二问）
35     seg_starts = [dates_u(1); m_dates + days(1)];
36     seg_ends = [m_dates - days(1); dates_u(end)];
37     slopes=[]; mid_months=[]; gain_mid=[]; gain_big=[];
38     for k=1:length(seg_starts)
39         mask = dates_u>=seg_starts(k) & dates_u<=seg_ends(k);
40         if sum(mask)<5, continue; end
41         t_vec=(0:sum(mask)-1)';
42         b=[ones(size(t_vec)),t_vec]\daily_per(mask);
43         slopes(end+1)=b(2);
44         mid_date = seg_starts(k)+days(round(sum(mask)/2));
45         mid_months(end+1)=month(mid_date);
46         if k>1
47             [~,tidx]=min(abs(dates_u-m_dates(k-1)));
48             dp=mean(daily_per(tidx:min(tidx+3,end))) - ...
49                 mean(daily_per(max(1,tidx-3):tidx));
50             if m_types(k-1)==1
51                 gain_mid(end+1)=dp;
52             else
53                 gain_big(end+1)=dp;
54             end
55         end
56     end
57     res_k0 = abs(mean(slopes));
58     if isnan(res_k0), res_k0=0.35; end
59     %%季节算子
60     S_vals = ones(1,4);
61     season_idx = zeros(size(mid_months));
62     for i=1:length(mid_months)
63         mv=mid_months(i);
64         season_idx(i)=get_s_idx(mv);
65     end
66     for s=1:4
67         if any(season_idx==s)
68             S_vals(s)=mean(slopes(season_idx==s))/mean(slopes);
69         end
70     end
71     %%损伤系数
72     a_mid=[]; a_big=[];
73     for i=2:length(slopes)
74         kp_prev = abs(slopes(i-1)/S_vals(season_idx(i-1)));
75         kp_curr = abs(slopes(i)/S_vals(season_idx(i)));
76         if kp_prev>1e-6
77             alpha=(kp_curr-kp_prev)/kp_prev;
78             if m_types(i-1)==1
79                 a_mid(end+1)=alpha;
80             else
81                 a_big(end+1)=alpha;

```

```

82         end
83     end
84 end
85 alpha_mid = max(0, mean(a_mid, 'omitnan'));
86 alpha_big = max(0, mean(a_big, 'omitnan'));
87 res_gm = max(5, mean(gain_mid, 'omitnan'));
88 res_gb = max(12, mean(gain_big, 'omitnan'));
89 %%回溯
90 back_len = days(dates_u(1)-t_born);
91 temp_P = daily_per(1);
92 p_back=zeros(back_len,1);
93 for i=1:back_len
94     dt=dates_u(1)-days(i);
95     s_idx=get_s_idx(month(dt));
96     k_t=res_k0*S_vals(s_idx);
97     gain=0;
98     idx=find(m_dates==dt,1);
99     if ~isempty(idx)
100         if m_types(idx)==1, gain=res_gm;
101         else, gain=res_gb; end
102     end
103     temp_P=temp_P + k_t - gain;
104     temp_P=min(101.5,temp_P);
105     p_back(i)=temp_P;
106 end
107 value_series=[flipud(p_back); daily_per];
108 %%GA 优化
109 lb=[40,150,0.05];
110 ub=[120,400,1.5];
111 options=optimoptions('ga','PopulationSize',30,...
112     'MaxGenerations',25,'Display','off');
113 bestX = ga(@(X) fitness_func(X,value_series,dates_u(end),...
114     res_k0,S_vals,res_gm,res_gb,...
115     alpha_mid,alpha_big,...
116     m_types,t_born),3,[],[],[],[],lb,ub,[],options);
117 %%获取最优解详情
118 [score,L_total,fail_date, v_final] = fitness_func(bestX,value_series,dates_u(end),...
119     res_k0,S_vals,res_gm,res_gb,...
120     alpha_mid,alpha_big,...
121     m_types,t_born);
122 %%获取维护次数
123 [~,~,~,~,nm,nb] = simulate(bestX, value_series, dates_u(end), ...
124     res_k0, S_vals, res_gm, res_gb, alpha_mid, alpha_big, m_types, t_born, ...
125     @(nm,nb) 300 + nm*3 + nb*12);
126 fprintf('最优策略: 中维护间隔: %d天 大维护间隔: %d天 速率阈值: %.3f\n',...
127     round(bestX(1)),round(bestX(2)),bestX(3));
128 fprintf('寿命 %.2f 年 | 报废 %s | 年均成本 %.2f | 中维%d次 大维%d次\n',...
129     L_total, datestr(fail_date), score, nm, nb);
130 final_table=[final_table;
131     {char(dev),L_total,score,round(bestX(1)),round(bestX(2)),bestX(3),datestr(fail_date),nm,
132     nb}];
133 % 保存基准参数供敏感性分析
134 dev_params{d}.device = char(dev);
135 dev_params{d}.value_series = value_series;
136 dev_params{d}.start_date = dates_u(end);
137 dev_params{d}.k0 = res_k0;
138 dev_params{d}.S_vals = S_vals;
139 dev_params{d}.gm = res_gm;
140 dev_params{d}.gb = res_gb;
141 dev_params{d}.alpha_m = alpha_mid;
142 dev_params{d}.alpha_b = alpha_big;
143 dev_params{d}.m_types = m_types;
144 dev_params{d}.t_born = t_born;
145 dev_params{d}.baseX = bestX;
146 dev_params{d}.L_base = L_total;
147 dev_params{d}.A0 = score;
148 dev_params{d}.nm0 = nm;
149 dev_params{d}.nb0 = nb;
150 % 绘图
151 t_axis = t_born + days(0:length(v_final)-1);
152 idx_plot = find(t_axis >= dates_u(1), 1);
153 figure('Name', [char(dev) ' 生命周期仿真'], 'Color', 'w'); hold on; grid on;
154 plot(t_axis(idx_plot:end), v_final(idx_plot:end), 'Color', '#a30543', 'LineWidth', 1.2, '
155     DisplayName', '瞬时透水量');
156 ma_val = movmean(v_final, [364 0]);
157 plot(t_axis(idx_plot:end), ma_val(idx_plot:end), 'Color', '#e6b919', 'LineWidth', 1.2, '
158     DisplayName', '年均值线');
159 yline(37, '--', 'Color', '#c9542c', 'Label', '寿命终止阈值(37%)', 'LineWidth', 1, 'DisplayName
160     ', '寿命终止阈值', 'LabelHorizontalAlignment', 'left');
161 title(['过滤器 ', char(dev), ' 全生命周期透水量变化图']);
162 xlabel('时间'); ylabel('透水量 (%)');
163 legend('Location', 'southwest');
164 datetick('x', 'yyyy-mm');

```

```

161 end
162 %%输出基准结果
163 disp('基准优化结果: ');
164 ResultTable = cell2table(final_table,...
165     'VariableNames',{'设备','寿命','年均成本','中维护间隔','大维护间隔','速率阈值','报废日期','
    中维次数','大维次数'});
166 disp(ResultTable);
167 save('optimization_results.mat', 'final_table', 'dev_params');
168 fprintf('\n敏感性分析: 成本波动下重优化\n');
169 scenarios = {
170     'Baseline', 1.0, 1.0, 1.0;
171     % 购价单因素 ±10%, ±15%
172     'Buy +10%', 1.10, 1.0, 1.0;
173     'Buy -10%', 0.90, 1.0, 1.0;
174     'Buy +15%', 1.15, 1.0, 1.0;
175     'Buy -15%', 0.85, 1.0, 1.0;
176     % 大中维护合并: 中维护与大维护同步 ±10%, ±15%
177     'Mid +10% Big +10%', 1.0, 1.10, 1.10;
178     'Mid -10% Big -10%', 1.0, 0.90, 0.90;
179     'Mid +15% Big +15%', 1.0, 1.15, 1.15;
180     'Mid -15% Big -15%', 1.0, 0.85, 0.85;
181     % 全因素同向 ±10%, ±15%
182     'All +10%', 1.10, 1.10, 1.10;
183     'All -10%', 0.90, 0.90, 0.90;
184     'All +15%', 1.15, 1.15, 1.15;
185     'All -15%', 0.85, 0.85, 0.85;
186     % 交叉扰动: 购价与维护反向
187     'Buy+10% Mid-10% Big-10%', 1.10, 0.90, 0.90;
188     'Buy-10% Mid+10% Big+10%', 0.90, 1.10, 1.10;
189     'Buy+15% Mid-15% Big-15%', 1.15, 0.85, 0.85;
190     'Buy-15% Mid+15% Big+15%', 0.85, 1.15, 1.15;
191 };
192 n_dev = length(dev_params);
193 sens_table = [];
194 options_sens = optimoptions('ga','PopulationSize',20,...
195     'MaxGenerations',15,'Display','off');
196 lb = [40,150,0.05];
197 ub = [120,400,1.5];
198 for d = 1:n_dev
199     dev = dev_params{d}.device;
200     fprintf('\n----- Sensitivity for device %s ----- \n', dev);
201     VS = dev_params{d}.value_series;
202     sd = dev_params{d}.start_date;
203     k0 = dev_params{d}.k0;
204     S_vals = dev_params{d}.S_vals;
205     gm = dev_params{d}.gm;
206     gb = dev_params{d}.gb;
207     am = dev_params{d}.alpha_m;
208     ab = dev_params{d}.alpha_b;
209     m_types = dev_params{d}.m_types;
210     t_born_dev = dev_params{d}.t_born;
211     L0 = dev_params{d}.L_base;
212     A0 = dev_params{d}.A0;
213     nm0 = dev_params{d}.nm0;
214     nb0 = dev_params{d}.nb0;
215     for sc = 1:size(scenarios,1)
216         label = scenarios{sc,1};
217         f_buy = scenarios{sc,2};
218         f_mid = scenarios{sc,3};
219         f_big = scenarios{sc,4};
220         C_fixed = 300*f_buy + nm0*3*f_mid + nb0*12*f_big;
221         A_fixed = C_fixed / L0;
222         cost_fun = @(nm,nb) 300*f_buy + nm*3*f_mid + nb*12*f_big;
223
224         fit_handle = @(X) fitness_cost_only(X, VS, sd, k0, S_vals, gm, gb, am, ab, ...
225             m_types, t_born_dev, cost_fun);
226         bestX_new = ga(fit_handle, 3, [], [], [], [], lb, ub, [], options_sens);
227         [A_opt, L_opt, ~, ~, nm_opt, nb_opt] = simulate(bestX_new, VS, sd, ...
228             k0, S_vals, gm, gb, am, ab, m_types, t_born_dev, cost_fun);
229         delta = (A_fixed - A_opt) / A_opt * 100;
230         if abs(delta) <= 3
231             feasible = 'Feasible';
232         elseif abs(delta) <= 5
233             feasible = 'Mild';
234         else
235             feasible = 'Adjust';
236         end
237         sens_table = [sens_table; {dev, label, L0, L_opt, A_fixed, A_opt, delta, feasible, ...
238             round(bestX_new(1)), round(bestX_new(2)), bestX_new(3)}];
239         fprintf(' %-12s: Afixed=%.2f, Aopt=%.2f, delta=%.2f%%, %s\n', ...
240             label, A_fixed, A_opt, delta, feasible);
241     end
242 end

```

```

243 %%函数部分
244 %适应度函数
245 function [score, L_total, fail_date, v_out] = fitness_func(X, value_series, start_date, ...
246     k0, S_vals, gm, gb, alpha_m, alpha_b, m_types, t_born)
247 cost_fun = @(nm,nb) 300 + nm*3 + nb*12;
248 [score, L_total, fail_date, v_out] = simulate(X, value_series, start_date, ...
249     k0, S_vals, gm, gb, alpha_m, alpha_b, m_types, t_born, cost_fun);
250 end
251 %返回适应度值供GA重优化的函数
252 function score = fitness_cost_only(X, value_series, start_date, ...
253     k0, S_vals, gm, gb, alpha_m, alpha_b, m_types, t_born, cost_fun)
254 score = simulate(X, value_series, start_date, ...
255     k0, S_vals, gm, gb, alpha_m, alpha_b, m_types, t_born, cost_fun);
256 end
257 %核心仿真函数
258 function [score, L_total, fail_date, v_out, nm, nb] = simulate(X, value_series, start_date, ...
259     k0, S_vals, gm, gb, alpha_m, alpha_b, m_types, t_born, cost_fun)
260 T_mid = round(X(1));
261 T_big = round(X(2));
262 r_mid = X(3);
263 curr_P = value_series(end);
264 curr_date = start_date;
265 nm = 0; nb = 0;
266 last_m = 0; last_b = 0;
267 day = 0;
268 h_win = value_series(max(1,end-364):end);
269 prev_avg = mean(h_win);
270 has_big = any(m_types==2);
271 emergency = false; below = 0;
272 while day < 6000
273     day = day + 1;
274     curr_date = curr_date + days(1);
275     s_idx = get_s_idx(month(curr_date));
276     D = k0 * S_vals(s_idx) * (1+alpha_m)^nm * (1+alpha_b)^nb;
277     curr_P = curr_P - D;
278     if length(value_series) > 7
279         rate = (curr_P - value_series(end-7)) / 7;
280     else
281         rate = 0;
282     end
283     do_big = (day - last_b >= T_big);
284     do_mid = ~do_big && ((day - last_m >= T_mid) || (rate <= -r_mid));
285     if do_big
286         curr_P = curr_P + gb; nb = nb + 1;
287         last_b = day; last_m = day;
288     elseif do_mid
289         curr_P = curr_P + gm; nm = nm + 1;
290         last_m = day;
291     end
292     value_series(end+1) = curr_P;
293     h_win(end+1) = curr_P;
294     if length(h_win) > 365, h_win(1) = []; end
295     curr_avg = mean(h_win);
296
297     if (prev_avg >= 37) && (curr_avg < 37)
298         if has_big && ~emergency
299             curr_P = curr_P + gb;
300             emergency = true;
301         end
302         below = 1;
303     elseif curr_avg < 37
304         below = below + 1;
305     else
306         below = 0;
307     end
308     if below >= 30 || curr_P <= 0
309         break;
310     end
311     prev_avg = curr_avg;
312 end
313 v_out = value_series;
314 fail_date = curr_date;
315 L_total = days(curr_date - t_born) / 365.25;
316 total_cost = cost_fun(nm, nb);
317 score = total_cost / L_total;
318 end
319 %辅助函数
320 function s = get_s_idx(mv)
321 if ismember(mv, [3 4 5])
322     s = 1;
323 elseif ismember(mv, [6 7 8])
324     s = 2;
325 elseif ismember(mv, [9 10 11])
326     s = 3;

```

```
327 else
328     s = 4;
329 end
330 end
```

code/Q3Q4.m